



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» Кафедра 806

Группа М80-407Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»

Профиль Информатика

Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Открытая микросервисная платформа для автоматизации тренировочного процесса и взаимодействия в системе «тренер-спортсмен»

Автор ВКРБ: _____ Калуцкий Максим Витальевич (_____)

Руководитель: _____ Булакина Мария Борисовна (_____)

К защите допустить

Заведующий кафедрой № 806 _____ Крылов Сергей Сергеевич (_____) _____

_____ мая 2026 года

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 70 страниц, 14 рисунков, 25 таблиц, 12 использованных источников, 6 приложений.

BACKEND-ПЛАТФОРМА, МИКРОСЕРВИСНАЯ АРХИТЕКТУРА, ТРЕНЕР, СПОРТСМЕН, ТРЕНИРОВОЧНЫЙ ПРОЦЕСС, REST API, NATS JETSTREAM, POSTGRES SQL, АНАЛИТИКА, AI-РЕКОМЕНДАЦИИ

Выпускная квалификационная работа посвящена разработке backend-части открытой микросервисной платформы CoachLink для автоматизации тренировочного процесса и взаимодействия в системе «тренер-спортсмен». В работе рассматривается проблема разрозненного обмена тренировочными заданиями и отчётами через мессенджеры, электронные таблицы и произвольные форматы данных.

Объектом исследования является процесс информационного взаимодействия между тренером и спортсменом при планировании, выполнении и анализе тренировок. Предметом исследования является backend-платформа для автоматизации назначения тренировочных заданий, сбора стандартизированных отчётов, отправки уведомлений и анализа данных тренировочного процесса.

Целью работы является разработка backend-части открытой микросервисной платформы для автоматизации тренировочного процесса и взаимодействия в системе «тренер-спортсмен».

В рамках работы спроектирована и реализована микросервисная архитектура из семи независимых сервисов. Сервисы реализованы на языке Go, используют PostgreSQL для хранения данных, NATS JetStream для событийного взаимодействия и OpenAPI для документирования публичных интерфейсов. Backend обеспечивает регистрацию пользователей, разграничение ролей тренера и спортсмена, управление связями и тренировочными группами, назначение тренировок, приём стандартизированных отчётов, формирование уведомлений, агрегацию тренировочных данных и генерацию AI-рекомендаций на основе истории тренировок.

Первая версия ориентирована на лёгкую атлетику, однако архитектура позволяет адаптировать тренировки, отчёты и шаблоны под другие виды спорта.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	7
ВВЕДЕНИЕ	8
1 АНАЛИТИЧЕСКИЙ ОБЗОР	11
1.1 Особенности взаимодействия тренера и спортсмена	11
1.2 Использование мессенджеров	11
1.3 Использование электронных таблиц	12
1.4 Спортивные трекеры и приложения	13
1.5 Специализированные коммерческие платформы	14
1.6 Сравнительный анализ подходов	15
1.7 Функциональные требования	16
1.7.1 Пользователи и роли	16
1.7.2 Связь тренер-спортсмен	17
1.7.3 Тренировочные группы	17
1.7.4 Тренировочные планы и назначения	17
1.7.5 Отчёты	18
1.7.6 Уведомления	18
1.7.7 Аналитика	18
1.7.8 AI-рекомендации	19
1.8 Нефункциональные требования	19
1.9 Выводы по главе 1	20
2 ПРОЕКТНАЯ ЧАСТЬ	21
2.1 Общий подход к проектированию	21
2.2 Высокоуровневая архитектура	21
2.3 Технологический стек	23
2.4 Состав сервисов	23
2.4.1 API Gateway	23
2.4.2 Auth Service	24
2.4.3 User Service	25
2.4.4 Training Service	26
2.4.5 Notification Service	27
2.4.6 Analytics Service	27
2.4.7 AI Service	28

2.5	Модель данных	28
2.5.1	Auth Service	28
2.5.2	User Service	29
2.5.3	Training Service	30
2.5.4	Notification Service	30
2.5.5	ER-диаграмма	31
2.6	Межсервисное взаимодействие	31
2.6.1	Синхронное взаимодействие	31
2.6.2	Асинхронное взаимодействие	32
2.7	Проектирование ключевых сценариев	33
2.7.1	Регистрация пользователя	33
2.7.2	Создание связи тренер-спортсмен	34
2.7.3	Назначение тренировки группе	35
2.7.4	Отправка отчёта	36
2.7.5	Получение AI-рекомендации	36
2.8	Проектирование публичного API	37
2.9	Расширяемость под другие виды спорта	38
2.10	Выводы по главе 2	38
3	РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ	40
3.1	Общая структура реализации	40
3.2	API Gateway	40
3.3	Auth Service	42
3.3.1	Регистрация	42
3.3.2	Вход и токены	42
3.4	User Service	43
3.4.1	Синхронизация профилей	43
3.4.2	Поиск пользователей	43
3.4.3	Заявки и связи	44
3.4.4	Тренировочные группы	44
3.4.5	Внутренние API	44
3.5	Training Service	45
3.5.1	Тренировочные планы	45
3.5.2	Назначения	46
3.5.3	Просроченность задания	46
3.5.4	Отчёты	47

3.5.5	Шаблоны тренировок	47
3.5.6	События Training Service	48
3.6	Notification Service	48
3.6.1	Обработка событий	48
3.6.2	API уведомлений	49
3.7	Analytics Service	50
3.7.1	Сводка спортсмена	50
3.7.2	Прогресс по периодам	50
3.7.3	Обзор тренера	51
3.8	AI Service	51
3.8.1	Проверка доступа	51
3.8.2	Получение данных	51
3.8.3	Формирование prompt	51
3.8.4	Генерация через Ollama	52
3.9	Тестирование	53
3.9.1	Уровни тестирования	53
3.9.2	Unit-тесты	53
3.9.3	E2E и интеграционные тесты	54
3.9.4	Таблица результатов	55
3.10	Матрица трассируемости требований	55
3.11	Наблюдаемость	57
3.12	Выводы по главе 3	57
	ЗАКЛЮЧЕНИЕ	59
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	62
	ПРИЛОЖЕНИЕ А Состав backend-сервисов	63
	ПРИЛОЖЕНИЕ Б Группы публичного API	64
	ПРИЛОЖЕНИЕ В Примеры JSON-запросов и ответов	65
	ПРИЛОЖЕНИЕ Г ER-диаграмма основных таблиц	68
	ПРИЛОЖЕНИЕ Д События NATS JetStream в backend-платформе	69
	ПРИЛОЖЕНИЕ Е Таблица тестирования	70

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

Backend-платформа — серверная часть программной системы, включающая бизнес-логику, API, хранение данных и межсервисное взаимодействие

Микросервис — независимый сервис с ограниченной зоной ответственности, взаимодействующий с другими сервисами через API или сообщения

Микросервисная архитектура — архитектурный подход, при котором система строится как набор независимых сервисов, развиваемых и разворачиваемых отдельно

Событийное взаимодействие — асинхронный способ обмена, при котором сервис публикует событие, а подписчики обрабатывают его независимо

Stateless-сервис — сервис, не владеющий собственной предметной базой данных и вычисляющий результат по входным данным и данным других сервисов

Публичный API — внешний программный интерфейс платформы, доступный клиентским приложениям через API Gateway

Внутренний API — служебный программный интерфейс межсервисного взаимодействия, недоступный внешним клиентам

API Gateway — единая точка входа в backend-платформу, выполняющая проверку доступа и маршрутизацию запросов к внутренним сервисам

Связь тренер-спортсмен — подтверждённое отношение между тренером и спортсменом, определяющее права доступа к тренировочным данным

Reverse proxy — сервер-посредник, принимающий запросы от клиентов и перенаправляющий их к внутренним сервисам, скрывая их адреса от внешних потребителей

Durable consumer — подписчик брокера сообщений, сохраняющий позицию чтения между перезапусками и обеспечивающий обработку пропущенных сообщений

Middleware — промежуточный обработчик в цепочке обработки HTTP-запроса, выполняющий сквозную логику (аутентификация, логирование, CORS) до передачи запроса основному обработчику

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

API — Application Programming Interface, программный интерфейс приложения

AI — Artificial Intelligence, искусственный интеллект

REST — Representational State Transfer, архитектурный стиль построения HTTP API

HTTP — HyperText Transfer Protocol, протокол передачи гипертекста

JWT — JSON Web Token, формат токена для передачи утверждений о пользователе

SQL — Structured Query Language, язык структурированных запросов к реляционным базам данных

DTO — Data Transfer Object, объект передачи данных между слоями или сервисами

UUID — Universally Unique Identifier, универсальный уникальный идентификатор

JSON — JavaScript Object Notation, текстовый формат обмена структурированными данными

JSONB — Binary JSON, бинарный тип хранения JSON-документов в PostgreSQL

CORS — Cross-Origin Resource Sharing, механизм контроля междоменных HTTP-запросов

NATS — система обмена сообщениями для событийного взаимодействия микросервисов

FCM — Firebase Cloud Messaging, сервис push-уведомлений

LLM — Large Language Model, большая языковая модель

E2E — End-to-End, сквозное тестирование пользовательского сценария

CI/CD — Continuous Integration/Continuous Delivery, непрерывная интеграция и доставка

БД — база данных

ВВЕДЕНИЕ

Современный тренировочный процесс всё чаще опирается не только на очное взаимодействие тренера и спортсмена, но и на постоянный обмен цифровыми данными. Тренер должен заранее формировать тренировочные задания, распределять их между спортсменами, учитывать индивидуальные особенности подготовки, получать обратную связь после выполнения тренировки и принимать решения о дальнейшем изменении нагрузки. При работе с одним спортсменом такие действия могут выполняться вручную и не требуют сложной информационной системы. Однако при увеличении количества спортсменов и появлении тренировочных групп ручной обмен сообщениями быстро становится неудобным и ненадёжным.

На практике взаимодействие тренера со спортсменами часто организуется с помощью мессенджеров, электронных таблиц и произвольных файлов. Такой подход не требует внедрения специализированного программного обеспечения, но создаёт ряд существенных проблем, подробно рассмотренных в первой главе. Особенно остро они проявляются в циклических видах спорта, где важно не только получить отдельный отчёт, но и накопить историю структурированных данных для оценки динамики подготовки.

В связи с этим актуальной является задача разработки единой backend-платформы, которая автоматизирует взаимодействие тренера и спортсмена, стандартизирует формат тренировочных отчётов, обеспечивает хранение истории тренировок и предоставляет программные интерфейсы для внешних клиентских приложений. Такая платформа должна решать не только задачу обмена сообщениями, но и задачу организации предметной логики тренировочного процесса: ролей пользователей, связей между тренером и спортсменами, групп, планов, назначений, отчётов, уведомлений и аналитики.

В данной работе рассматривается backend-часть платформы CoachLink. Backend реализует серверную логику, хранение данных, публичные REST API, межсервисное взаимодействие и интеграцию с инфраструктурными компонентами. Мобильное приложение рассматривается только как внешний клиент, который обращается к backend через API Gateway. Разработка мобильного приложения не входит в область ответственности данной выпускной квалификационной работы.

Платформа CoachLink проектируется как расширяемая система для автоматизации тренировочного процесса в различных видах спорта. В целях упрощения разработки прототипа текущая реализация ориентирована исключительно на лёгкую атлетику. При этом архитектура платформы остаётся универсальной: базовые сущности (пользователь, тренер, спортсмен, группа, план, назначение, отчёт, уведомление) не имеют жёсткой привязки к конкретному виду спорта и в дальнейшем могут быть расширены за счёт добавления специфических атрибутов и шаблонов для других дисциплин.

Объектом исследования является процесс информационного взаимодействия между тренером и спортсменом при планировании, выполнении и анализе тренировок.

Предметом исследования является backend-платформа для автоматизации назначения тренировочных заданий, сбора стандартизированных отчётов, отправки уведомлений и анализа данных тренировочного процесса.

Целью выпускной квалификационной работы является разработка backend-части открытой микросервисной платформы для автоматизации тренировочного процесса и взаимодействия в системе «тренер-спортсмен».

Для достижения поставленной цели необходимо решить следующие задачи:

1. провести анализ предметной области и существующих программных решений, выявить их ограничения в контексте автоматизации взаимодействия тренера и спортсмена;
2. сформулировать функциональные и нефункциональные требования к backend-платформе;
3. спроектировать микросервисную архитектуру системы, разработать модель данных и спецификации программных интерфейсов;
4. реализовать спроектированные микросервисы и обеспечить их взаимодействие;
5. провести тестирование backend-компонентов и оценить соответствие разработанной системы исходным требованиям.

При выполнении работы используются методы системного анализа, объектно-ориентированного и компонентного проектирования, проектирования REST API, проектирования реляционных баз данных и тестирования программного обеспечения. Выбор микросервисной

архитектуры соотносится с подходами к построению распределённых систем, описанными в работах по шаблонам микросервисов, проектированию независимых сервисов и переходу к cloud-native архитектурам [1; 2; 3; 4].

Теоретическая значимость работы заключается в проектировании модели backend-платформы, объединяющей микросервисный подход, событийное взаимодействие, стандартизированный сбор тренировочных данных и аналитическую обработку этих данных. Работа демонстрирует, как предметная область тренировочного процесса может быть представлена через набор независимых сервисов с чёткими зонами ответственности.

Практическая значимость заключается в создании открытой серверной основы для платформы CoachLink. Реализованный backend может использоваться как база для дальнейшего развития системы: подключения мобильного приложения, добавления новых видов спорта, расширения аналитики, интеграции со спортивными устройствами и развития AI-сценариев. Благодаря открытому API систему можно адаптировать под разные клиентские приложения и внешние интеграции.

Выпускная квалификационная работа состоит из введения, трёх глав, заключения и приложений.

В первой главе проводится аналитический обзор существующих подходов к организации взаимодействия тренера и спортсмена. Рассматриваются мессенджеры, электронные таблицы, спортивные трекеры и специализированные коммерческие платформы. На основе сравнения формулируются требования к backend-платформе.

Во второй главе описывается проектирование CoachLink. Рассматриваются архитектура микросервисов, модель данных, межсервисное взаимодействие, ключевые пользовательские сценарии и выбор технологического стека.

В третьей главе описывается практическая реализация спроектированных микросервисов и результаты тестирования.

В заключении подводятся итоги работы, оценивается достижение цели и формулируются направления дальнейшего развития платформы.

1 АНАЛИТИЧЕСКИЙ ОБЗОР

1.1 Особенности взаимодействия тренера и спортсмена

Тренировочный процесс представляет собой повторяющийся цикл планирования, выполнения, отчётности и корректировки нагрузки. Тренер формирует задание, спортсмен выполняет его и передаёт обратную связь, после чего тренер оценивает состояние спортсмена и принимает решение о следующем этапе подготовки. Для эффективной работы важно, чтобы этот цикл был регулярным, прозрачным и основанным на достоверных данных.

В классической форме взаимодействие тренера и спортсмена может происходить очно: тренер объясняет задание на тренировке, наблюдает за выполнением и сразу корректирует действия спортсмена. Однако в реальных условиях такая модель не всегда достаточна. Спортсмены могут выполнять часть тренировок самостоятельно, находиться в разных местах, тренироваться в разное время или входить в разные группы. Поэтому тренеру необходим цифровой инструмент, позволяющий сопровождать тренировочный процесс вне очных занятий.

Цифровая поддержка тренировочного процесса должна решать несколько задач:

1. передача тренировочных заданий спортсменам;
2. управление индивидуальными и групповыми назначениями;
3. получение отчётов о выполнении заданий;
4. хранение истории тренировок;
5. уведомление участников о важных событиях;
6. анализ накопленных данных;
7. разграничение ролей и прав доступа.

Если рассматривать систему только как средство отправки сообщений, можно ограничиться мессенджером. Но при необходимости хранить историю тренировок, связывать отчёт с конкретным заданием и выполнять аналитику простого обмена сообщениями становится недостаточно. Требуется система, в которой данные имеют заранее определённую структуру.

1.2 Использование мессенджеров

Мессенджеры выступают наиболее распространённым инструментом

цифрового взаимодействия благодаря своей массовости и интуитивно понятному интерфейсу, не требующему времени на освоение. Данный класс приложений позволяет тренеру оперативно создавать групповые чаты, рассылать тренировочные задания или консультировать спортсменов в индивидуальном порядке. В свою очередь, спортсмен получает возможность быстро предоставлять обратную связь, используя различные форматы данных.

Главное преимущество мессенджеров — низкий порог входа. Для начала работы не требуется разрабатывать отдельную систему, создавать аккаунты в специализированном сервисе или обучать пользователей. Мессенджер удобен для оперативного общения и решения организационных вопросов.

Однако при использовании мессенджеров для ведения тренировочного процесса возникают существенные ограничения.

Ключевым ограничением является отсутствие стандартизированной структуры отчёта: спортсмен сам решает, что написать и в каком формате, в результате чего тренер получает разноформатные данные, непригодные для автоматической обработки.

Помимо этого, отчёты трудно связывать с конкретными заданиями. Если спортсмен прислал сообщение спустя несколько дней после нескольких назначений, тренер вынужден вручную восстанавливать соответствие, что особенно неудобно при групповой работе.

Существенной проблемой остаётся смешение тренировочных данных с общей перепиской: организационные сообщения, личные вопросы и отчёты оказываются в одном чате, что затрудняет поиск и систематизацию истории тренировок.

Наконец, мессенджеры не предоставляют аналитических инструментов: рассчитать процент выполнения заданий, динамику нагрузки или суммарную дистанцию без ручной обработки данных невозможно.

Следовательно, мессенджер эффективен как канал общения, но не является полноценной платформой для управления тренировочным процессом.

1.3 Использование электронных таблиц

Электронные таблицы частично решают проблему структуры данных. Тренер может создать таблицу с колонками для даты, задания, длительности, дистанции, пульса и комментария. Спортсмены могут заполнять строки после

выполнения тренировок, а тренер — просматривать данные и выполнять простые расчёты.

Преимущество таблиц заключается в том, что они позволяют хранить историю тренировок в более структурированном виде, чем мессенджеры. Таблицы также поддерживают фильтрацию, сортировку, формулы и визуализацию. Для небольших групп этого может быть достаточно.

Однако у таблиц также есть ограничения.

Ключевым ограничением является отсутствие полноценной ролевой модели: доступ можно настроить лишь на уровне документа или листа, но выразить предметные правила — кто видит только свои задания, кто видит всю группу — средствами таблицы затруднительно.

Вместе с тем таблицы не моделируют жизненный цикл задания: они хранят данные, но не отражают переходы между состояниями «назначено», «выполнено», «в архиве», что делает контроль за выполнением непрозрачным.

Дополнительную сложность создаёт отсутствие встроенного механизма уведомлений: тренер не получает системного оповещения об отправленном отчёте, а спортсмен может не заметить новое задание без дополнительного сообщения в мессенджере.

Наконец, при росте числа спортсменов таблица становится трудноуправляемой: возникает риск случайного изменения чужих данных, удаления строк и нарушения согласованности формата.

Таким образом, электронные таблицы лучше мессенджеров подходят для хранения истории, но не обеспечивают полноценную автоматизацию взаимодействия.

1.4 Спортивные трекеры и приложения

Спортивные трекеры и приложения предназначены для фиксации фактических показателей активности. Они могут автоматически собирать дистанцию, темп, пульс, маршрут, высоту, каденс и другие параметры. Для лёгкой атлетики такие инструменты полезны, потому что позволяют получить объективные данные о тренировке.

Сильной стороной спортивных трекеров является автоматизация измерений. Спортсмену не нужно вручную вводить все показатели, если устройство уже записало тренировку. Также многие приложения предоставляют графики и базовую аналитику.

Однако спортивные трекеры не полностью решают задачу взаимодействия тренера и спортсмена.

Ключевым ограничением является ориентированность трекеров на фиксацию факта активности, а не на управление назначениями: логика «задание назначено — выполнено — отчёт привязан к назначению» в большинстве таких сервисов отсутствует или реализована в ограниченном виде.

Помимо этого, трекеры, как правило, не учитывают организационный контекст тренера: тренировочные группы, индивидуальные планы, заявки на связь, ролевой доступ и уведомления о новых отчётах выходят за рамки их функциональности.

Существенным препятствием служит и закрытость большинства подобных сервисов: их API может быть ограничен, платным или недоступным для самостоятельного расширения, что исключает возможность интеграции в открытую платформу.

Наконец, не все виды тренировочной работы поддаются автоматической фиксации. Силовая подготовка, технические упражнения и восстановительные занятия требуют текстового комментария и субъективной оценки нагрузки, которые трекер не собирает.

Следовательно, спортивные трекеры полезны как потенциальный источник данных, но не заменяют backend-платформу взаимодействия тренера и спортсмена.

1.5 Специализированные коммерческие платформы

На рынке существуют специализированные платформы для тренеров, которые позволяют назначать тренировки, отслеживать выполнение, работать с группами и анализировать данные. К наиболее распространённым относятся TrainingPeaks, Final Surge и CoachNow. Такие решения ближе всего к требованиям предметной области.

Их преимуществами являются готовый пользовательский интерфейс, наличие функциональности для тренеров, интеграции с устройствами и развитая аналитика. Однако даже у таких платформ есть ряд принципиальных ограничений.

Ключевым из них является закрытость API и отсутствие возможности самостоятельной доработки: адаптировать платформу под специфические

требования конкретного вида спорта или интегрировать её с внешними инструментами, как правило, невозможно.

Не менее значимой проблемой выступает зависимость от внешнего поставщика: пользователь не контролирует архитектуру, хранение данных, правила доступа и дальнейшее развитие продукта, что создаёт риски при изменении условий использования или прекращении работы сервиса.

Наконец, большинство профессиональных платформ ориентированы на коммерческое использование и недоступны для небольших групп или начинающих тренеров.

Таким образом, хотя функционал специализированных платформ доказывает востребованность подобных решений, их проприетарный характер не удовлетворяет требованиям расширяемости и независимости от поставщика, что обосновывает необходимость разработки собственной открытой системы.

1.6 Сравнительный анализ подходов

Сравнительный анализ проведён по критериям, важным для автоматизации тренировочного процесса. Результаты сравнения приведены в таблице 1.

Таблица 1 – Сравнение подходов к организации взаимодействия тренера и спортсмена

Критерий	Мессендж.	Эл. табл.	Трекеры	Коммерч. платформы	CoachLink
Единый формат отчётов	Нет	Частично	Да, для активности	Да	Да
Связь отчёта с заданием	Нет	Частично	Ограниченно	Да	Да
Групповые назначения	Частично	Частично	Ограниченно	Да	Да
Роли тренера и спортсмена	Нет	Ограниченно	Частично	Да	Да
Уведомления о событиях	Да, но не профильные	Нет	Да	Да	Да
Хранение истории	Да, в разном формате	Да	Да	Да	Да

Продолжение таблицы 1

Критерий	Мессендж.	Эл. табл.	Трекеры	Коммерч. платформы	CoachLink
Аналитика	Нет	Базовая	Да, по активности	Да	Да
Открытость API	Нет	Ограниченно	Зависит от сервиса	Зависит от сервиса	Да
Возможность доработки	Низкая	Средняя	Низкая	Низк./сред.	Высокая
Самостоятельное развёртывание	Нет	Частично	Нет	Обычно нет	Да

Анализ таблицы показывает, что ни один из распространённых подходов полностью не удовлетворяет требованиям открытой backend-платформы. Мессенджеры и таблицы удобны как вспомогательные инструменты, но не обеспечивают предметную модель. Спортивные трекеры полезны как источник данных, но не покрывают весь процесс взаимодействия. Специализированные коммерческие платформы функциональны, но ограничены с точки зрения открытости и самостоятельного развития.

CoachLink ориентируется на сочетание нескольких свойств: открытость, структурированность данных, ролевой доступ, групповые назначения, уведомления, аналитика и расширяемость. Это делает разработку собственной backend-платформы обоснованной.

1.7 Функциональные требования

На основе проведённого анализа предметной области можно сформулировать следующие функциональные требования к разрабатываемой системе.

1.7.1 Пользователи и роли

Подсистема управления пользователями должна обеспечивать:

- регистрацию и аутентификацию пользователей;
- поддержку ролевой модели доступа с двумя базовыми ролями: «тренер» и «спортсмен»;
- разграничение прав доступа к API-методам в зависимости от роли

пользователя.

Такой подход позволяет изолировать функции, доступные разным категориям пользователей. Например, только тренер может создавать тренировочные планы и группы, а отправлять отчёт по назначению может только спортсмен, которому это назначение принадлежит.

1.7.2 Связь тренер-спортсмен

Подсистема управления связями между тренером и спортсменом должна обеспечивать:

- отправку спортсменом заявки на установление связи с тренером;
- возможность принятия или отклонения входящей заявки тренером;
- ограничение, при котором один спортсмен может быть связан не более чем с одним тренером одновременно.

Механизм подтверждаемых заявок защищает тренера от автоматического добавления спортсменов и позволяет явно контролировать состав подопечных. Ограничение связи в первой версии системы упрощает логику доступа и однозначно определяет тренера, ответственного за тренировочный процесс спортсмена.

1.7.3 Тренировочные группы

Подсистема управления тренировочными группами должна обеспечивать:

- создание и редактирование тренером тренировочных групп;
- добавление спортсменов в группу только при наличии установленной связи с данным тренером.

Объединение спортсменов в группы необходимо для массового назначения одинаковых тренировочных планов нескольким спортсменам одновременно.

1.7.4 Тренировочные планы и назначения

Подсистема планирования тренировок должна обеспечивать:

- создание тренером тренировочных планов;
- назначение тренировочных планов отдельным спортсменам или тренировочным группам;

- просмотр списка назначений;
- поддержку статусов назначений, позволяющих отличать активные, выполненные и архивные задания;
- сохранение описаний тренировок как шаблонов для повторного использования.

Данный функционал автоматизирует основной цикл предметной области: постановку тренировочного задания, его выполнение спортсменом и последующий контроль со стороны тренера.

1.7.5 Отчёты

Подсистема отчётности должна обеспечивать:

- отправку спортсменом структурированного отчёта по конкретному тренировочному назначению;
- хранение отчёта в едином формате;
- поддержку полей отчёта, необходимых для первой версии системы: текстовый комментарий, длительность, субъективная нагрузка, дистанция и показатели пульса.

Единый формат отчётов устраняет разрозненность данных и делает собранную информацию пригодной для последующей автоматической обработки.

1.7.6 Уведомления

Подсистема уведомлений должна обеспечивать:

- создание уведомлений о важных событиях: заявке на связь, принятии заявки, назначении тренировки, удалении задания, отправке отчёта, добавлении или удалении из группы;
- хранение уведомлений в базе данных;
- предоставление уведомлений пользователям через API.

Система уведомлений обеспечивает своевременное информирование участников о ключевых событиях без необходимости постоянного мониторинга приложения.

1.7.7 Аналитика

Подсистема аналитики должна обеспечивать:

- агрегацию данных из отчётов спортсмена;
- расчёт сводных показателей за выбранный период: количества отчётов, суммарной длительности, суммарной дистанции, средней субъективной нагрузки и средних показателей пульса;
- вычисление процента выполненных назначений.

Агрегированные показатели позволяют тренеру оценивать динамику нагрузки и эффективность тренировочного процесса на основе накопленных структурированных данных.

1.7.8 AI-рекомендации

Подсистема интеллектуальной поддержки должна обеспечивать генерацию текстовой рекомендации по конкретному спортсмену на основе последних отчётов и сводной статистики.

AI-модуль рассматривается как вспомогательный инструмент поддержки принятия решений и не заменяет экспертную оценку тренера.

1.8 Нефункциональные требования

Помимо функциональных возможностей, backend-платформа должна удовлетворять ряду нефункциональных требований, связанных с архитектурой, сопровождением и эксплуатацией системы. К ним относятся:

1. модульность архитектуры и возможность независимого развития основных компонентов системы;
2. наличие единого программного интерфейса для взаимодействия внешних клиентов с backend-платформой;
3. документирование публичных API-контрактов в машиночитаемом формате;
4. логическое разделение данных между функциональными областями системы;
5. надёжное хранение структурированных данных с поддержкой транзакционности;
6. поддержка асинхронной обработки событий между компонентами системы;
7. воспроизводимое локальное развёртывание системы;
8. возможность расширения предметной модели под другие виды

- спорта;
9. тестируемость бизнес-логики;
 10. наблюдаемость работы сервисов: возможность количественно отслеживать частоту запросов, долю ошибок и задержки в процессе эксплуатации;
 11. минимальная зависимость от конкретного клиентского приложения.

1.9 Выводы по главе 1

В первой главе были рассмотрены основные способы организации цифрового взаимодействия тренера и спортсмена: мессенджеры, электронные таблицы, спортивные трекеры и специализированные коммерческие платформы. Анализ показал, что каждый из подходов решает только часть задачи.

Мессенджеры удобны для общения, но не обеспечивают структурированное хранение данных. Электронные таблицы позволяют хранить историю, но не обладают полноценной предметной логикой, уведомлениями и ролевой моделью. Спортивные трекеры собирают данные активности, но не организуют полный цикл назначения и отчётности. Специализированные коммерческие платформы функциональны, но ограничены с точки зрения открытости, самостоятельного развёртывания и доработки.

На основании анализа сформулированы требования к backend-платформе CoachLink. Система должна поддерживать роли пользователей, связи тренера и спортсмена, тренировочные группы, планы, назначения, стандартизированные отчёты, уведомления, аналитику и AI-рекомендации. Для удовлетворения этих требований в следующей главе предлагается микросервисная архитектура backend-платформы.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Общий подход к проектированию

Целью проектной части является построение backend-архитектуры, которая покрывает основные сценарии взаимодействия тренера и спортсмена и при этом остаётся расширяемой. Система должна быть пригодна для развития: добавления новых видов спорта, подключения внешних клиентов, появления дополнительных аналитических модулей и интеграций со спортивными устройствами.

Для решения этой задачи выбран микросервисный подход. Он позволяет разделить систему на независимые компоненты по предметным областям. В CoachLink такими областями являются авторизация, профили и связи пользователей, тренировочный процесс, уведомления, аналитика и AI-рекомендации.

Монолитная архитектура для первой версии могла бы быть проще в реализации, однако она хуже демонстрирует независимость предметных модулей и сложнее масштабируется организационно. В системе «тренер-спортсмен» отдельные части имеют разную природу. Например, сервис авторизации отвечает за безопасность и токены, сервис тренировок — за предметную логику назначений и отчётов, сервис уведомлений — за реакцию на события, а сервис аналитики — за расчёт показателей. Разделение этих частей повышает ясность архитектуры.

Проектирование выполнялось с учётом следующих принципов:

1. единая точка входа для внешних клиентов;
2. разделение ответственности между сервисами;
3. собственная база данных для stateful-сервисов;
4. использование событий для асинхронных операций;
5. использование внутренних API для синхронных межсервисных запросов;
6. документирование публичного API;
7. возможность расширения предметной модели.

2.2 Высокоуровневая архитектура

Внешний клиент взаимодействует с платформой через API Gateway.

Клиентом может быть мобильное приложение, web-интерфейс или другой сервис. В рамках данной работы клиентская часть не рассматривается как результат разработки, а используется только как потребитель REST API.

API Gateway проверяет авторизацию и перенаправляет запросы к backend-сервисам. Сервисы реализованы на Go и взаимодействуют между собой через HTTP и NATS JetStream. Для хранения данных используется PostgreSQL. Для генерации AI-рекомендаций используется локальный LLM-сервер Ollama. На рисунке 1 представлена компонентная схема backend-платформы. Сплошными стрелками на схеме показаны синхронные взаимодействия: HTTP-запросы между сервисами и обращения к базам данных. Пунктирными стрелками показано асинхронное событийное взаимодействие через NATS JetStream.

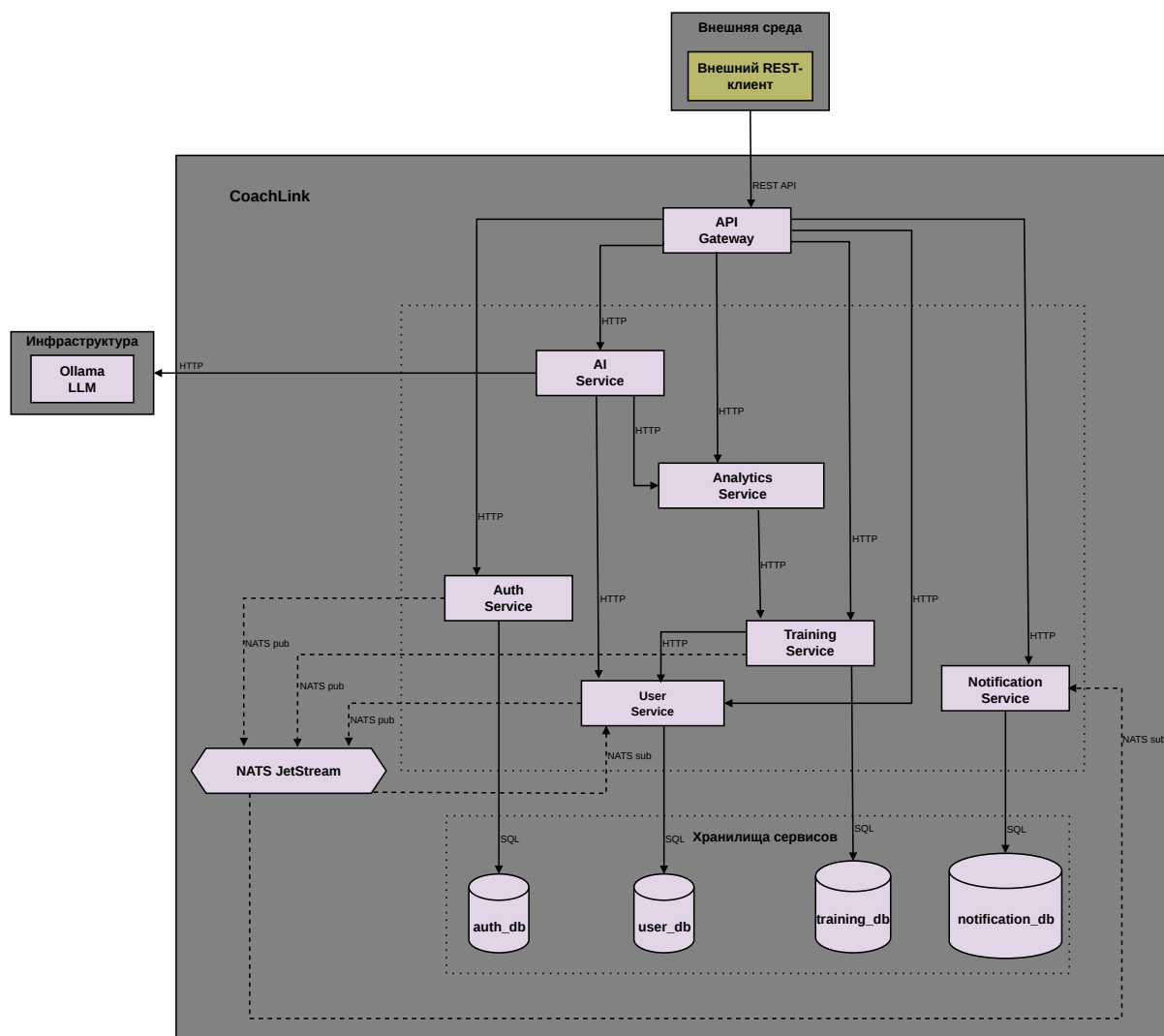


Рисунок 1 – Компонентная схема backend-платформы

В такой архитектуре API Gateway не содержит бизнес-логики. Его задача — принять запрос, проверить JWT, добавить служебные заголовки и передать запрос в нужный сервис. Предметные правила остаются внутри соответствующих сервисов.

2.3 Технологический стек

Выбор языка Go [5] в качестве основного инструмента backend-разработки обусловлен его высокой эффективностью при создании микросервисной архитектуры. К ключевым преимуществам языка в контексте решаемой задачи относятся: эффективная реализация параллелизма, высокая скорость исполнения кода и богатая экосистема модулей для маршрутизации запросов, взаимодействия с базами данных и брокерами сообщений.

Для HTTP API используется Echo [6]. Этот фреймворк предоставляет маршрутизацию, middleware, обработку запросов и удобную интеграцию с валидацией. Для доступа к PostgreSQL [7] применяется `sqlx`, так как он позволяет сохранять контроль над SQL-запросами и при этом упрощает маппинг результатов в структуры Go. Миграции базы данных выполняются через `goose`.

Для аутентификации используется схема access/refresh токенов. Access token реализован как JWT [8] и содержит идентификатор пользователя, логин и роль. Пароли хранятся в виде bcrypt-хешей. Для асинхронного взаимодействия сервисов используется NATS JetStream [9], обеспечивающий durable-подписки и доставку событий после временной недоступности сервиса.

Контейнеризация выполнена через Docker Compose [10]. Это позволяет запускать все сервисы платформы, PostgreSQL, NATS и Ollama [11] одной командой. API документируется в формате OpenAPI [12], что делает backend-контракты пригодными для подключения внешних клиентов.

2.4 Состав сервисов

2.4.1 API Gateway

API Gateway является входной точкой для всех публичных HTTP-запросов. Он решает следующие задачи:

- маршрутизация запросов по префиксам;

- проверка JWT access token;
- извлечение идентификатора пользователя, роли и логина из токена;
- проброс пользовательского контекста во внутренние сервисы через заголовки;
- обработка CORS;
- предоставление health-check endpoint.

Публичные маршруты без авторизации ограничены регистрацией, входом и обновлением токена:

- POST /api/v1/auth/register;
- POST /api/v1/auth/login;
- POST /api/v1/auth/refresh.

Остальные маршруты требуют заголовок `Authorization: Bearer <token>`. После успешной проверки Gateway устанавливает заголовки:

- X-User-ID;
- X-User-Role;
- X-User-Login.

Такой подход реализует паттерн API Gateway, описанный в работах по микросервисной архитектуре [1]: аутентификация выполняется один раз на входе, а внутренние сервисы доверяют Gateway и используют переданные заголовки для авторизации. Это исключает необходимость передавать JWT-секрет каждому сервису и упрощает добавление новых сервисов в систему.

2.4.2 Auth Service

Auth Service отвечает за учётные данные и токены. В его область ответственности входят:

- регистрация пользователя;
- валидация логина, email, пароля и роли;
- хеширование пароля;
- вход пользователя;
- выпуск access token и refresh token;
- обновление пары токенов;
- отзыв refresh token при logout;
- публикация события регистрации.

Access token реализован как JWT. В payload токена включаются поля, представленные на рисунке 2.

```
1 {  
2   "sub": "uuid пользователя",  
3   "login": "user_login",  
4   "role": "coach | athlete",  
5   "exp": 1234567890,  
6   "iat": 1234567890  
7 }
```

Рисунок 2 – Структура payload access token

Refresh token хранится в базе данных не в открытом виде, а в виде SHA-256 хеша. Это снижает риск компрометации токена при утечке базы данных. При refresh старый токен удаляется, а пользователю выдаётся новая пара токенов.

После успешной регистрации Auth Service публикует событие `coachlink.user.registered`. Это событие используется User Service для создания пользовательского профиля.

2.4.3 User Service

User Service отвечает за доменную информацию о пользователях. Он не хранит пароли и не выпускает токены. Его задачи:

- хранение профилей пользователей;
- синхронизация профилей из события регистрации;
- поиск пользователей по ФИО и логину;
- управление заявками спортсмена тренеру;
- управление связями тренер-спортсмен;
- управление тренировочными группами;
- предоставление внутренних API для других сервисов.

Связь тренера и спортсмена создаётся через заявку. Это решение выбрано для явного контроля доступа: спортсмен не может автоматически прикрепиться к тренеру без подтверждения. Тренер видит входящие заявки и может принять или отклонить их.

Тренировочные группы принадлежат тренеру. Добавлять в группу можно только спортсменов, которые уже связаны с этим тренером. Это предотвращает

ситуацию, когда тренер назначает тренировку спортсмену, не входящему в его область ответственности.

2.4.4 Training Service

Training Service реализует центральную предметную логику платформы. Он отвечает за:

- создание тренировочных планов;
- назначение планов спортсменам;
- назначение планов группе;
- получение списка назначений;
- фильтрацию назначений;
- удаление и архивирование назначений;
- отправку отчётов;
- хранение шаблонов тренировок;
- предоставление внутренних API для аналитики.

Тренировочный план и назначение спроектированы как отдельные сущности, связанные отношением «один ко многим». План содержит название, описание и дату тренировки, а назначение связывает план с конкретным спортсменом и хранит индивидуальный статус выполнения. Такое разделение необходимо для групповых тренировок: тренер создаёт один план, а система формирует отдельное назначение для каждого участника группы. Денормализация ФИО и логина спортсмена в таблице назначений позволяет фильтровать и отображать списки без дополнительных запросов в User Service, что устраняет проблему N+1 запросов при получении списка назначений тренера.

Назначение имеет статус:

- `assigned` — задание назначено и ожидает отчёта;
- `completed` — спортсмен отправил отчёт;
- `archived` — тренер перенёс выполненное задание в архив.

Просроченность не хранится как отдельное поле, а вычисляется при чтении. Это уменьшает риск рассинхронизации данных. Назначение считается просроченным, если оно находится в статусе `assigned`, а дата выполнения уже прошла.

2.4.5 Notification Service

Notification Service реализует паттерн событийного подписчика: он не вызывается напрямую другими сервисами, а подписывается на события NATS JetStream и реагирует на них созданием записей в таблице `notifications`. Для каждого типа события создаётся отдельный `durable consumer`, что обеспечивает независимую обработку и позволяет восстановить пропущенные сообщения после перезапуска сервиса.

Сервис обрабатывает следующие события:

- заявка спортсмена тренеру;
- принятие или отклонение заявки;
- назначение тренировки;
- удаление задания;
- отправка отчёта;
- добавление спортсмена в группу;
- удаление спортсмена из группы.

Пользователь может получить список уведомлений, отфильтровать их по признаку прочтения, получить количество непрочитанных уведомлений в ответе списка и отметить уведомления как прочитанные. Также сервис хранит FCM-токены устройств. Если `Firebase credentials` не настроены, `push`-доставка отключается, но уведомления продолжают сохраняться в базе данных.

2.4.6 Analytics Service

Analytics Service является `stateless`-сервисом. Он не имеет собственной базы данных, а получает данные из Training Service через внутренние API. Такое решение исключает проблему синхронизации аналитической базы с основной базой тренировок и гарантирует, что результаты агрегации всегда вычисляются по актуальным данным. При необходимости масштабирования аналитика может быть вынесена в отдельное хранилище без изменения публичного API.

Сервис предоставляет три основных представления данных:

- сводку по спортсмену;
- прогресс по неделям или месяцам;
- обзор по спортсменам тренера.

Каждое представление формируется из набора ключевых метрик

тренировочного процесса:

- количество отчётов;
- суммарная длительность;
- средняя длительность;
- суммарная дистанция;
- средняя воспринимаемая нагрузка;
- средний пульс;
- максимальный пульс;
- количество назначений;
- процент выполнения.

2.4.7 AI Service

AI Service отвечает за генерацию рекомендаций по тренировочному процессу конкретного спортсмена на основе накопленных данных.

Публичный

endpoint:

POST /api/v1/ai/athletes/{id}/recommendations.

Сервис доступен только тренеру. Перед генерацией он проверяет через User Service, что спортсмен действительно связан с текущим тренером. Затем AI Service получает из Analytics Service сводную статистику и последние отчёты спортсмена. Количество отчётов ограничено пятью, чтобы prompt оставался компактным. Типовой сценарий — рекомендации по последним 3-5 тренировкам.

Генерация выполняется через Ollama. Prompt составляется на русском языке и содержит конкретные данные: объём тренировок, длительность, дистанцию, субъективную нагрузку, пульс и комментарии спортсмена. AI-модуль является вспомогательным инструментом и не заменяет решение тренера.

2.5 Модель данных

2.5.1 Auth Service

Auth Service использует две основные таблицы (таблица 3).

Таблица 3 – Таблицы Auth Service и их назначение

Таблица	Назначение
<code>users</code>	Учётные данные пользователя, роль и пароль в виде bcrypt-хеша
<code>refresh_tokens</code>	Хеши refresh-токенов и срок их действия

Таблица `users` содержит логин, email, ФИО, роль, хеш пароля и временные метки. Роль ограничена значениями `coach` и `athlete` через CHECK-констрейнт, что исключает появление невалидных ролей на уровне базы данных. Логин имеет ограничение уникальности. Таблица `refresh_tokens` связана с пользователем через внешний ключ с каскадным удалением: при удалении пользователя все его refresh-токены удаляются автоматически.

2.5.2 User Service

User Service использует следующие таблицы (таблица 4).

Таблица 4 – Таблицы User Service и их назначение

Таблица	Назначение
<code>user_profiles</code>	Профили пользователей, синхронизированные из Auth Service
<code>connection_requests</code>	Заявки спортсмена тренеру
<code>coach_athlete_relations</code>	Активные связи тренер-спортсмен
<code>training_groups</code>	Тренировочные группы тренера
<code>training_group_members</code>	Состав тренировочных групп

Разделение `users` и `user_profiles` отражает разделение ответственности сервисов: Auth Service хранит учётные данные, User Service — доменную информацию для поиска, связей и групп. Таблица `coach_athlete_relations` использует UNIQUE-констрейнт на `athlete_id`, что ограничивает спортсмена одним тренером и упрощает логику авторизации в Training Service и Analytics Service. Для текстового поиска по ФИО и логину используются trigram-индексы расширения `pg_trgm`,

обеспечивающие поиск подстрок и автодополнение.

2.5.3 Training Service

Training Service использует таблицы, описанные в таблице 5.

Таблица 5 – Таблицы Training Service и их назначение

Таблица	Назначение
training_plans	Общая информация о тренировочном плане
training_assignments	Назначение плана конкретному спортсмену
training_reports	Отчёт спортсмена по назначению
training_templates	Шаблоны тренировок тренера

Разделение плана и назначения на уровне таблиц соответствует проектному решению, описанному в разделе 2.3: один план порождает несколько назначений при групповой тренировке. Таблица `training_assignments` содержит внешний ключ на план с каскадным удалением, CHECK-констрейнт на статус и денормализованные поля ФИО спортсмена. Отчёт связан с назначением через UNIQUE-ограничение: каждому назначению соответствует не более одного отчёта. Поле `perceived_effort` ограничено диапазоном от 0 до 10 через CHECK-констрейнт.

2.5.4 Notification Service

Notification Service хранит данные в таблицах, представленных в таблице 6.

Таблица 6 – Таблицы Notification Service и их назначение

Таблица	Назначение
notifications	Уведомления пользователей
device_tokens	FCM-токены устройств

В уведомлении хранится тип, заголовок, текст, дополнительные данные в JSONB, признак прочтения и дата создания. JSONB-поле позволяет добавлять к уведомлению контекст: идентификатор заявки, задания или группы.

2.5.5 ER-диаграмма

Общая структура данных представлена на ER-диаграмме (рисунок 3). Диаграмма отражает логические связи между основными сущностями платформы. В микросервисной архитектуре таблицы физически распределены по базам данных отдельных сервисов. Расширенная ER-диаграмма с детализацией полей и типов данных приведена в приложении Г.

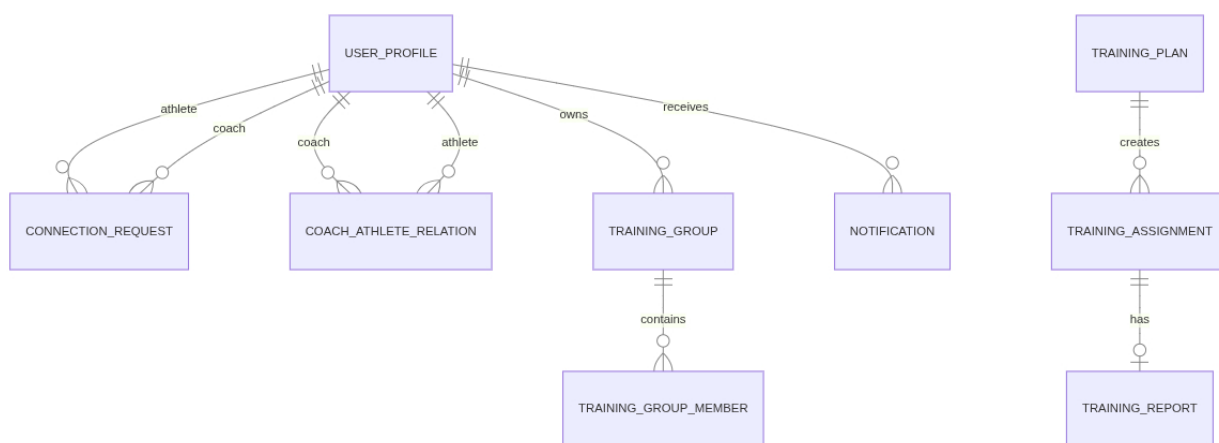


Рисунок 3 – ER-диаграмма основных сущностей платформы CoachLink

2.6 Межсервисное взаимодействие

2.6.1 Синхронное взаимодействие

Синхронные HTTP-вызовы используются в случаях, когда результат нужен для выполнения текущего запроса.

Примеры:

1. Training Service обращается к User Service при назначении плана группе, чтобы получить список участников группы;
2. Training Service получает профиль спортсмена, чтобы сохранить имя и логин в назначении;
3. Analytics Service обращается к Training Service за отчётами и агрегатами;
4. AI Service обращается к User Service для проверки связи тренера и спортсмена;
5. AI Service обращается к Analytics Service за сводкой и последними

отчётами.

Внутренние API не предназначены для внешних клиентов и доступны только сервисам внутри backend-платформы. При недоступности вызываемого сервиса вызывающий сервис возвращает клиенту ошибку с соответствующим HTTP-статусом. Повторные попытки для синхронных запросов не применяются, поскольку клиент ожидает ответ и повтор увеличил бы время ожидания. Для операций, допускающих задержку, используется асинхронное взаимодействие через NATS JetStream, где гарантия доставки обеспечивается брокером сообщений.

2.6.2 Асинхронное взаимодействие

Асинхронное взаимодействие реализовано через NATS JetStream. Оно используется там, где действие может быть выполнено после основного запроса и не должно блокировать пользователя. Все события платформы публикуются в единый стрим `COACHLINK` с wildcard-подпиской `coachlink.>` и файловым хранилищем, обеспечивающим сохранность сообщений при перезапуске брокера. Стрим создаётся программно при старте каждого сервиса через вызов `AddStream`, а не через внешнюю конфигурацию.

Для каждого типа события подписчик создаёт отдельный `durable consumer` с параметрами `DeliverAll` и `AckExplicit`, что обеспечивает гарантию доставки «at-least-once»: сообщение считается обработанным только после явного подтверждения (`Ack`), а при ошибке отклоняется (`Nak`) и будет доставлено повторно.

Например, при создании тренировочного назначения `Training Service` публикует событие `coachlink.training.assigned`. `Notification Service` получает событие и создаёт уведомление для спортсмена. Если `Notification Service` временно недоступен, `durable consumer` сохраняет позицию чтения, и событие будет обработано после восстановления сервиса.

В таблице 7 приведён список ключевых событий. Полный перечень событий с развёрнутым описанием назначения приведён в приложении Д.

Таблица 7 – Ключевые события NATS JetStream в backend-платформе

Событие	Источник	Получатель	Назначение
coachlink.user.registered	Auth Service	User Service	Создание профиля
coachlink.connection.requested	User Service	Notification Service	Уведомление тренера
coachlink.connection.accepted	User Service	Notification Service	Уведомление спортсмена
coachlink.connection.rejected	User Service	Notification Service	Уведомление спортсмена
coachlink.group.athlete_added	User Service	Notification Service	Уведомление спортсмена
coachlink.group.athlete_removed	User Service	Notification Service	Уведомление спортсмена
coachlink.training.assigned	Training Service	Notification Service	Уведомление спортсмена
coachlink.training.deleted	Training Service	Notification Service	Уведомление спортсмена
coachlink.report.submitted	Training Service	Notification Service	Уведомление тренера

2.7 Проектирование ключевых сценариев

2.7.1 Регистрация пользователя

Сценарий регистрации включает несколько шагов: клиент отправляет запрос через API Gateway, Auth Service валидирует данные, хеширует пароль, создаёт пользователя и публикует событие в NATS JetStream, после чего User Service создаёт профиль. На рисунке 4 показана последовательность взаимодействий при регистрации.

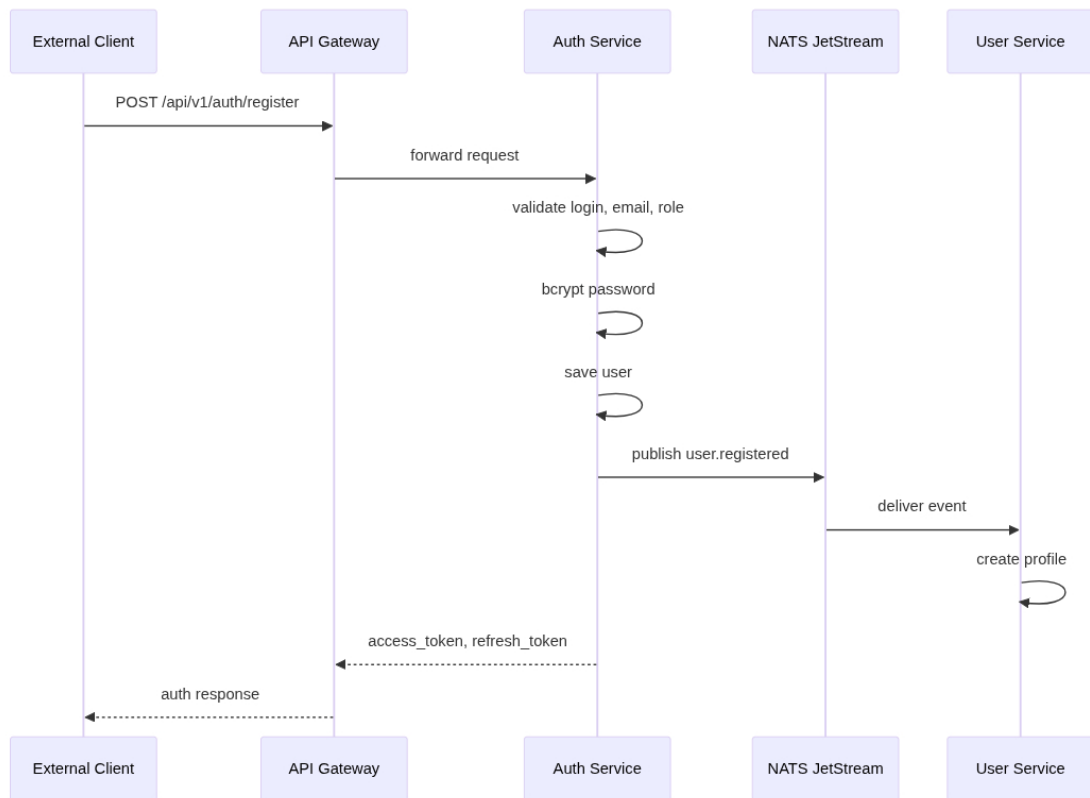


Рисунок 4 – Последовательность взаимодействий при регистрации пользователя

2.7.2 Создание связи тренер-спортсмен

Для установления связи спортсмен отправляет заявку тренеру. Тренер получает уведомление и принимает или отклоняет заявку. При принятии создаётся запись в таблице связей. На рисунке 5 представлена последовательность взаимодействий при создании связи.

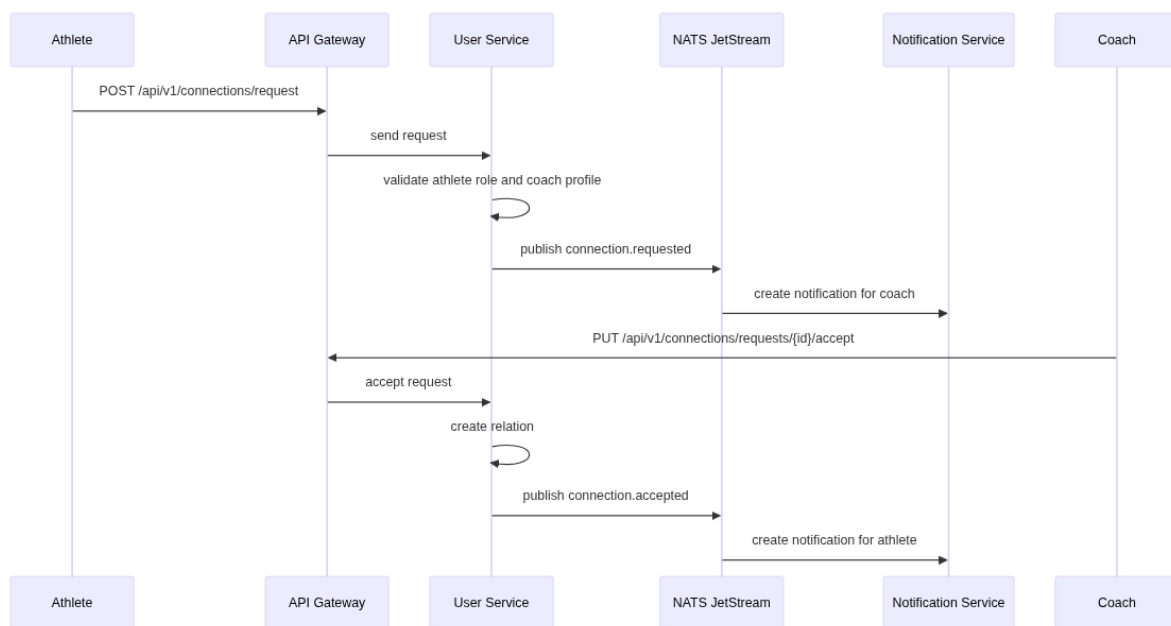


Рисунок 5 – Последовательность взаимодействий при создании связи тренер-спортсмен

2.7.3 Назначение тренировки группе

При назначении тренировки группе Training Service обращается к User Service для получения списка участников, затем создаёт отдельное назначение для каждого спортсмена и публикует событие для Notification Service. На рисунке 6 показана последовательность взаимодействий при назначении тренировки группе.

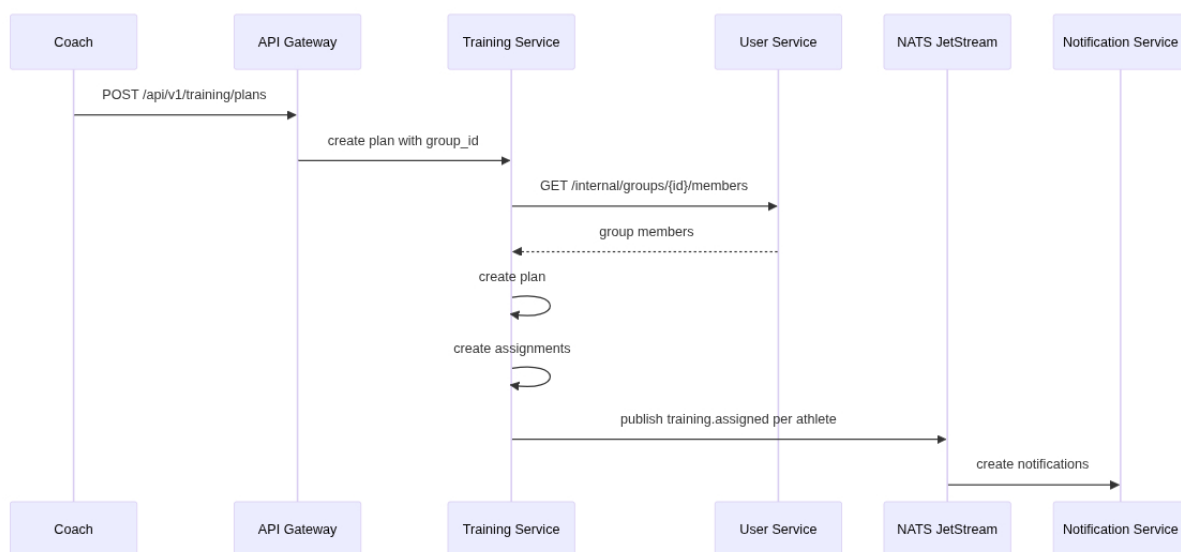


Рисунок 6 – Последовательность взаимодействий при назначении тренировки группе

2.7.4 Отправка отчёта

При отправке отчёта спортсмен передаёт данные через API Gateway в Training Service. Сервис проверяет принадлежность назначения, сохраняет отчёт, меняет статус назначения и публикует событие, на которое реагирует Notification Service. На рисунке 7 показана последовательность взаимодействий при отправке отчёта.

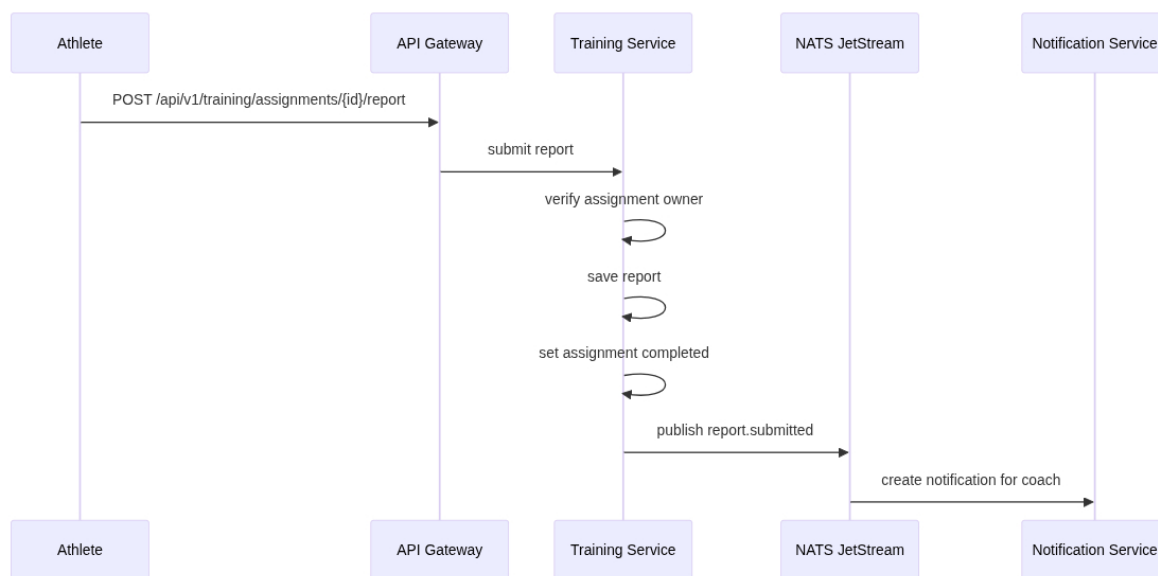


Рисунок 7 – Последовательность взаимодействий при отправке отчёта

2.7.5 Получение AI-рекомендации

Для получения AI-рекомендации тренер отправляет запрос через API Gateway в AI Service. Сервис проверяет связь тренера и спортсмена через User Service, получает статистику и последние отчёты из Analytics Service, формирует prompt и обращается к Ollama для генерации текста. На рисунке 8 показана последовательность взаимодействий при получении AI-рекомендации.

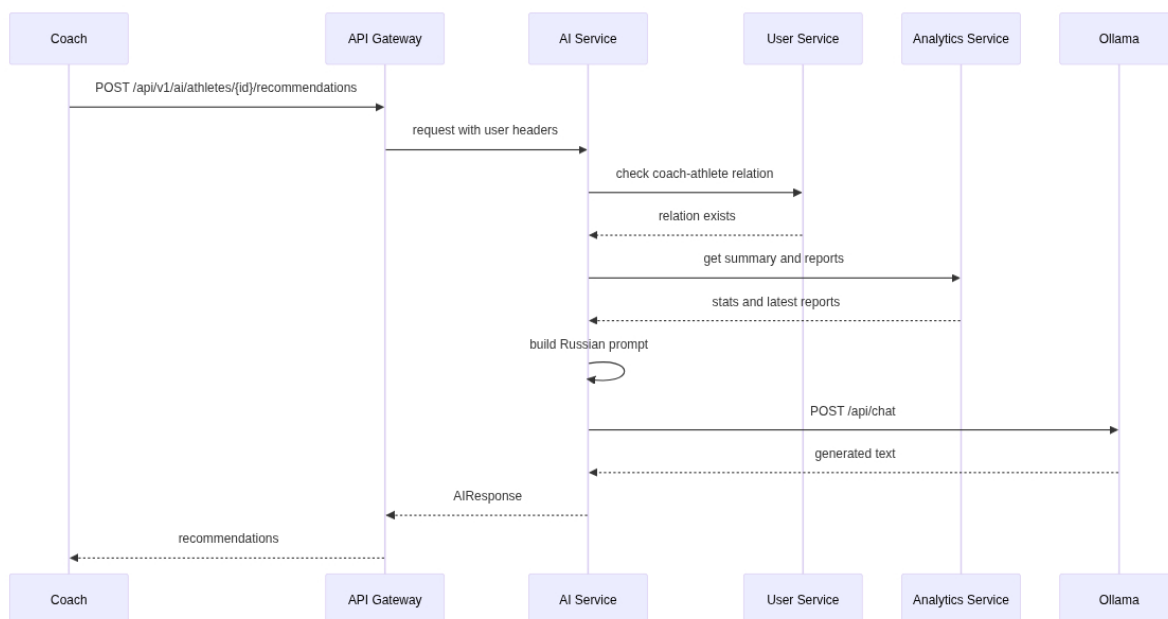


Рисунок 8 – Последовательность взаимодействий при получении AI-рекомендации

2.8 Проектирование публичного API

Публичный API разделён на группы по предметным областям (таблица 9).

Таблица 9 – Группы публичного API CoachLink

Группа	Назначение
/api/v1/auth/*	Регистрация, вход, обновление и отзыв токенов
/api/v1/users/*	Профиль текущего пользователя и поиск
/api/v1/connections/*	Заявки и связи тренер-спортсмен
/api/v1/groups/*	Тренировочные группы
/api/v1/training/*	Планы, назначения, отчёты, шаблоны
/api/v1/notifications/*	Уведомления
/api/v1/analytics/*	Аналитика тренировок
/api/v1/ai/*	AI-рекомендации

Такое разделение упрощает маршрутизацию на уровне API Gateway и позволяет документировать каждую группу endpoint независимо. При

проектировании API соблюдались следующие принципы: версионирование через префикс `/v1/`, что позволяет вводить несовместимые изменения в будущих версиях без нарушения работы существующих клиентов; единообразный формат ошибок с HTTP-статусом и JSON-телом, содержащим описание ошибки; пагинация списков через query-параметры `limit` и `offset`; использование стандартных HTTP-методов в соответствии с семантикой REST. Полная спецификация API оформлена в формате OpenAPI и доступна в репозитории проекта. Примеры endpoint каждой группы приведены в приложении Б.

2.9 Расширяемость под другие виды спорта

Хотя первая версия ориентирована на лёгкую атлетику, архитектура спроектирована с учётом расширения на другие виды спорта. Базовые сущности платформы — пользователи, роли, связи, группы, назначения, уведомления и аналитические агрегаты — не зависят от конкретного вида спорта и остаются неизменными при добавлении новых дисциплин.

Расширение предметной модели сосредоточено в Training Service. Конкретный механизм состоит в добавлении sport-specific полей в структуру тренировочного плана и отчёта. Например, для силовых видов спорта потребуются параметры подходов, повторений и веса, для плавания — стиль и длина бассейна, для велоспорта — средняя мощность и набор высоты. Эти поля могут быть реализованы через расширение схемы таблиц или через JSONB-поле с динамической структурой. При этом общий жизненный цикл назначения (создание, выполнение, отчёт, архивация), система уведомлений, аналитическая агрегация и API Gateway не требуют изменений.

2.10 Выводы по главе 2

В проектной части была предложена микросервисная backend-архитектура CoachLink. Архитектура включает API Gateway, сервис авторизации, сервис пользователей, сервис тренировок, сервис уведомлений, сервис аналитики и AI-сервис рекомендаций. Для хранения данных используется PostgreSQL, для событийного взаимодействия — NATS JetStream, для локального развёртывания — Docker Compose.

Спроектированная архитектура охватывает все функциональные

требования, сформулированные в аналитической главе: она поддерживает ролевую модель, связи тренера и спортсмена, групповые назначения, стандартизированные отчёты, уведомления и аналитику, а каждая предметная область закреплена за отдельным сервисом. При этом система остаётся расширяемой: текущая реализация ориентирована на лёгкую атлетику, но базовая модель может быть адаптирована для других видов спорта. Соответствие конкретных требований реализующим их endpoint и подтверждающим тестам приводится в главе реализации после описания API и тестирования.

3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ

3.1 Общая структура реализации

Backend-платформа CoachLink реализована как набор Go-сервисов в монорепозитории. Выбор монорепозитория обусловлен упрощением управления общими зависимостями, единой системой сборки и упрощённой координацией изменений между сервисами на этапе активной разработки.

Каждый сервис реализован как самостоятельный Go-модуль со стандартной внутренней структурой: точка входа, конфигурация, HTTP-обработчики, сервисный слой с бизнес-логикой, модели данных и слой доступа к базе данных или внешним клиентам. Общий код, используемый несколькими сервисами, вынесен в разделяемый пакет — в частности, типы и названия событий NATS JetStream, что обеспечивает согласованность событийных контрактов между публикующими и подписанными сервисами.

Для локального развёртывания используется Docker Compose: все семь сервисов, PostgreSQL, NATS JetStream и Ollama запускаются как контейнеры. Для каждого сервиса предусмотрен отдельный Dockerfile, обеспечивающий воспроизводимую сборку. Stateful-сервисы применяют SQL-миграции при запуске с помощью библиотеки `goose`, что гарантирует актуальность схемы базы данных без ручного вмешательства. Конфигурация каждого сервиса задаётся через переменные окружения, что упрощает развёртывание в различных средах. Краткое описание назначения каждого сервиса и его взаимодействий приведено в приложении А.

3.2 API Gateway

API Gateway реализован как отдельный Go-сервис на основе фреймворка `Echo`. Проксирование запросов выполняется через `net/http/httputil.ReverseProxy` из стандартной библиотеки Go. Для каждого backend-сервиса создаётся отдельный экземпляр `reverse proxy` с пользовательским `Director`, который сохраняет полный путь запроса и подменяет только `scheme` и `host` целевого сервиса. При ошибке проксирования пользовательский `ErrorHandler` возвращает клиенту ответ 502 в формате JSON.

Входящие запросы проходят через цепочку `middleware` в фиксированном

порядке: восстановление после паники, генерация идентификатора запроса, обработка CORS, структурированное логирование и проверка JWT. Маршрутизация выполняется по префиксам, как показано в таблице 11.

Таблица 11 – Маршрутизация API Gateway по префиксам

Префикс	Целевой сервис
/api/v1/auth/*	Auth Service
/api/v1/users/*	User Service
/api/v1/connections/*	User Service
/api/v1/groups/*	User Service
/api/v1/training/*	Training Service
/api/v1/notifications/*	Notification Service
/api/v1/analytics/*	Analytics Service
/api/v1/ai/*	AI Service

Gateway выполняет JWT-проверку для всех защищённых маршрутов. Исключениями являются регистрация, вход, обновление токена и health-check. После успешной проверки токена Gateway извлекает из полей токена идентификатор пользователя, роль и логин. Эти значения передаются дальше через заголовки, представленные на рисунке 9.

```
1 X-User-ID: <user_id>
2 X-User-Role: <coach|athlete>
3 X-User-Login: <login>
```

Рисунок 9 – Заголовки запроса, передаваемые Gateway внутренним сервисам

Такой механизм позволяет внутренним сервисам принимать решения о доступе без повторной проверки JWT. Например, Training Service проверяет, может ли текущий пользователь создать план, отправить отчёт или получить назначение.

Gateway также отвечает за обработку CORS-заголовков, необходимых для корректной работы web-клиентов, и предоставляет health-check endpoint для проверки доступности сервиса.

3.3 Auth Service

Auth Service отвечает за регистрацию, вход и управление токенами. Сервис использует PostgreSQL-базу `auth_db` и таблицы `users` и `refresh_tokens`.

3.3.1 Регистрация

При регистрации пользователь передаёт логин, email, пароль, ФИО и роль. DTO запроса содержит ограничения:

- логин обязателен, длина от 3 до 50 символов;
- email должен иметь корректный формат;
- пароль обязателен, длина от 8 до 128 символов;
- ФИО обязательно;
- роль должна быть `coach` или `athlete`.

Дополнительно сервис проверяет формат логина регулярным выражением (рисунок 10).

1 `^[a-zA-Z0-9-]+$`

Рисунок 10 – Регулярное выражение для проверки формата логина

Это ограничение исключает пробелы, кириллицу и специальные символы в логине, что упрощает поиск и использование логина в API.

Пароль хешируется через `bcrypt` с `cost`-фактором 12, что обеспечивает баланс между безопасностью и временем вычисления хеша. В базе хранится только `password_hash`, исходный пароль не сохраняется. Значение `cost`-фактора задаётся через переменную окружения и может быть изменено без модификации кода. После создания пользователя сервис публикует событие `coachlink.user.registered`, содержащее идентификатор пользователя, логин, ФИО, email и роль.

3.3.2 Вход и токены

При входе сервис ищет пользователя по логину и сравнивает пароль с `bcrypt`-хешем. При успешной проверке создаётся пара токенов:

- access token — JWT, подписанный алгоритмом HMAC-SHA256, со

сроком жизни 15 минут;

- refresh token — случайный UUID со сроком жизни 30 дней.

Структура полей access token описана в разделе проектирования (рисунок 2).

Refresh token не хранится в открытом виде. Сервис вычисляет SHA-256 хеш токена и сохраняет только хеш. Такой подход снижает риск компрометации при утечке базы данных: даже получив содержимое таблицы, злоумышленник не сможет восстановить исходный токен. При refresh старый хеш удаляется из базы, затем создаётся новая пара access/refresh. При logout refresh token также удаляется, что делает невозможным дальнейшее обновление сессии. Auth Service предоставляет четыре публичных endpoint: регистрацию, вход, обновление токена и выход.

3.4 User Service

User Service хранит пользовательские профили, связи тренер-спортсмен и тренировочные группы. Он использует базу `user_db`.

3.4.1 Синхронизация профилей

Профиль пользователя создаётся не напрямую при регистрации, а через событие `coachlink.user.registered`. Auth Service публикует событие после успешной регистрации, а User Service подписан на него через NATS JetStream. При получении события User Service создаёт запись в таблице `user_profiles`.

Такое разделение позволяет Auth Service отвечать только за учётные данные и токены, а User Service — за доменную информацию, используемую в тренировочном процессе.

3.4.2 Поиск пользователей

Сервис предоставляет поиск по ФИО и логину. Поиск используется, например, когда спортсмен ищет тренера для отправки заявки или тренер ищет спортсмена. Минимальная длина поискового запроса — два символа. Это ограничение снижает нагрузку на индекс при поиске по коротким строкам.

Для ускорения поиска в базе включено расширение `pg_trgm`, а для полей имени и логина создаются trigram-индексы. Это подходит для поиска подстрок

и автодополнения.

3.4.3 Заявки и связи

Связь тренера и спортсмена создаётся через заявку. Основной сценарий:

1. спортсмен находит тренера;
2. спортсмен отправляет заявку;
3. тренер получает уведомление;
4. тренер принимает или отклоняет заявку;
5. при принятии создаётся запись в `coach_athlete_relations`.

В первой версии у спортсмена может быть один тренер. Это ограничение задаётся уникальностью `athlete_id` в таблице связей. Такое решение упрощает логику доступа: Training Service и Analytics Service могут однозначно определять, какой тренер отвечает за спортсмена.

3.4.4 Тренировочные группы

Тренер может создавать тренировочные группы и добавлять туда своих спортсменов. Группа используется для массового назначения тренировочного плана: при создании плана с указанием `group_id` Training Service запрашивает состав группы через внутренний API User Service и формирует назначения для каждого участника.

Перед добавлением спортсмена в группу сервис проверяет наличие записи в таблице `coach_athlete_relations`. Если связь отсутствует, запрос отклоняется. При удалении группы записи в таблице участников удаляются каскадно. При добавлении или удалении спортсмена из группы публикуются события `coachlink.group.athlete_added` и `coachlink.group.athlete_removed`, на которые реагирует Notification Service.

3.4.5 Внутренние API

User Service предоставляет внутренние API для других сервисов (таблица 12).

Таблица 12 – Внутренние API User Service

Endpoint	Использование
GET /internal/users/{userId}	Получение профиля пользователя
GET /internal/groups/ {groupId}/members	Получение состава группы
GET /internal/coach/ {coachId}/has-athlete/ {athleteId}	Проверка связи тренера и спортсмена

Эти API не предназначены для внешних клиентов. Они позволяют Training Service, Analytics Service и AI Service выполнять предметные проверки и получать необходимые данные.

3.5 Training Service

Training Service является центральным сервисом тренировочного процесса. Он использует базу `training_db`.

3.5.1 Тренировочные планы

Тренировочный план содержит название, описание, дату и идентификатор тренера. План может быть назначен:

- одному или нескольким спортсменам по списку `athlete_ids`;
- всем спортсменам группы через `group_id`;
- комбинации группы и отдельных спортсменов.

Если указан `group_id`, Training Service обращается к User Service и получает участников группы. Участники группы и спортсмены из `athlete_ids` объединяются через ассоциативный массив с дедупликацией по идентификатору: если спортсмен указан одновременно в группе и в списке, он получит только одно назначение. Затем создаётся один тренировочный план и набор назначений для каждого спортсмена. При этом используется стратегия частичного успеха: если создание назначения для одного из спортсменов завершается ошибкой, остальные назначения продолжают создаваться.

3.5.2 Назначения

Назначение представляет связь между тренировочным планом и конкретным спортсменом. В назначении хранятся:

- `plan_id`;
- `athlete_id`;
- `coach_id`;
- имя и логин спортсмена;
- имя и логин тренера;
- статус;
- время назначения;
- время выполнения;
- время архивации.

Денормализация имени и логина спортсмена используется для фильтрации списков назначений без дополнительного запроса в User Service.

Статусы назначений приведены в таблице 14.

Таблица 14 – Статусы тренировочных назначений

Статус	Значение
<code>assigned</code>	Задание назначено и ожидает отчёта
<code>completed</code>	Спортсмен отправил отчёт
<code>archived</code>	Тренер перенёс выполненное назначение в архив

3.5.3 Просроченность задания

Просроченность вычисляется функцией `ComputeIsOverdue`. Задание считается просроченным, если:

- статус равен `assigned`;
- дата тренировки уже прошла.

Для определения просроченности используется `grace`-период: задание считается просроченным не в момент наступления даты тренировки, а спустя 24 часа после неё. Это даёт спортсмену возможность отправить отчёт в течение дня после тренировки. Просроченность не хранится в базе данных как отдельное поле, а вычисляется при каждом чтении. Это исключает необходимость фоновых задач и ситуацию, когда сохранённое значение

устаревает.

3.5.4 Отчёты

Перед созданием отчёта выполняется двойная проверка: сервис убеждается, что назначение находится в статусе `assigned`, и что для данного назначения ещё не существует отчёта. Если отчёт уже был отправлен ранее, возвращается ошибка `409 Conflict`. Такая двойная защита предотвращает дублирование отчётов даже при повторных запросах. После успешного создания отчёта статус назначения меняется на `completed`, а в поле `completed_at` записывается текущее время.

Формат отчёта представлен на рисунке 11.

```
1 {  
2   "content": "Тренировка прошла хорошо, последние интервалы дались  
   тяжело",  
3   "duration_minutes": 75,  
4   "perceived_effort": 7,  
5   "max_heart_rate": 188,  
6   "avg_heart_rate": 162,  
7   "distance_km": 12.4  
8 }
```

Рисунок 11 – Структура JSON-тела запроса на отправку отчёта

Поля отчёта выбраны с учётом первой реализации для лёгкой атлетики. Они позволяют фиксировать как объективные показатели, так и субъективную оценку нагрузки.

3.5.5 Шаблоны тренировок

Шаблоны тренировок позволяют тренеру сохранять часто используемые описания тренировок для повторного применения. Шаблон хранит те же предметные поля, что и план — название и описание, — но не привязан к дате и спортсменам. При создании тренировочного плана тренер может указать флаг `save_as_template`, и система автоматически сохранит копию описания как шаблон. В дальнейшем при создании нового плана тренер может выбрать существующий шаблон, и его поля будут использованы как основа. Каждый

шаблон принадлежит конкретному тренеру и недоступен другим пользователям, что обеспечивается проверкой `coach_id` на уровне сервисного слоя.

3.5.6 События Training Service

Training Service публикует события, перечисленные в таблице 15.

Таблица 15 – События Training Service

Событие	Когда публикуется
<code>coachlink.training.assigned</code>	Создано назначение
<code>coachlink.training.deleted</code>	Назначение удалено
<code>coachlink.report.submitted</code>	Спортсмен отправил отчёт

Эти события используются Notification Service для создания уведомлений.

3.6 Notification Service

Notification Service отвечает за создание, хранение и предоставление уведомлений пользователям. Сервис подписан на события NATS JetStream и реагирует на них созданием записей в базе `notification_db`. Благодаря событийной модели сервисы-источники событий не зависят от способа доставки уведомлений: Training Service публикует событие о назначении тренировки, а Notification Service самостоятельно решает, как уведомить спортсмена.

3.6.1 Обработка событий

Для каждого типа события создаётся отдельный `durable consumer` в NATS JetStream. Durable consumer сохраняет позицию чтения, что позволяет сервису продолжить обработку после перезапуска и не потерять события, опубликованные во время простоя.

Примеры соответствия событий и уведомлений представлены в таблице 16.

Таблица 16 – Соответствие событий и создаваемых уведомлений

Событие	Получатель	Тип уведомления
<code>connection.requested</code>	Тренер	Новая заявка от спортсмена
<code>connection.accepted</code>	Спортсмен	Заявка принята
<code>training.assigned</code>	Спортсмен	Новое тренировочное задание
<code>report.submitted</code>	Тренер	Новый отчёт
<code>group.athlete_added</code>	Спортсмен	Добавление в группу

Если уведомление успешно создано, сообщение подтверждается через `Ack`. Если произошла ошибка, сообщение отклоняется через `Nak`, чтобы оно могло быть обработано повторно.

3.6.2 API уведомлений

Notification Service предоставляет:

- получение списка уведомлений с пагинацией;
- фильтрацию по признаку прочтения;
- получение количества непрочитанных уведомлений;
- пометку одного уведомления прочитанным;
- пометку всех уведомлений прочитанными;
- регистрацию FCM-токена устройства.

FCM-интеграция является опциональной: если `credentials` не настроены, `push`-доставка отключается, но серверные уведомления продолжают сохраняться в базе данных и доступны через API. Это позволяет вести локальную разработку и тестирование без внешних Firebase-зависимостей. При наличии FCM-конфигурации `push`-уведомления отправляются в отдельной горутине и не блокируют создание серверного уведомления. В `push`-данные включаются тип события и идентификатор уведомления, что позволяет мобильному клиенту выполнять маршрутизацию к нужному экрану при нажатии на `push`. Для отправки на несколько устройств одного пользователя используется метод `SendEachForMulticast` из `Firebase Admin SDK`.

3.7 Analytics Service

Analytics Service реализован как stateless-сервис без собственной базы данных. При каждом запросе он обращается к Training Service через внутренние HTTP API, получает необходимые данные и выполняет агрегацию при обработке запроса. Такой подход исключает проблему рассинхронизации аналитической базы с основной базой тренировок: результат всегда вычисляется по актуальным данным. Сервис предоставляет три основных представления: сводку по спортсмену, прогресс по периодам и обзор для тренера.

3.7.1 Сводка спортсмена

Сводка представляет собой агрегированный набор показателей за весь период тренировок спортсмена. Она позволяет тренеру оценить общий объём работы, регулярность выполнения заданий и средние значения ключевых метрик. Сводка включает:

- количество отчётов;
- суммарную длительность;
- среднюю длительность;
- суммарную дистанцию;
- среднюю воспринимаемую нагрузку;
- средний пульс;
- максимальный пульс;
- количество назначений;
- процент выполнения.

3.7.2 Прогресс по периодам

Помимо общей сводки, тренеру важно отслеживать изменения показателей во времени. Analytics Service группирует отчёты по неделям или месяцам в зависимости от параметра запроса. Для каждого периода рассчитываются:

- количество отчётов;
- суммарная длительность;
- средняя нагрузка;
- средний пульс;

- суммарная дистанция.

Такая группировка позволяет выявлять тенденции в нагрузке и сравнивать периоды между собой, а не ограничиваться итоговыми значениями за всё время.

3.7.3 Обзор тренера

Для тренера сервис формирует обзор по всем его спортсменам: общее количество спортсменов, суммарное число назначений и отчётов, а также средний процент выполнения заданий. Этот endpoint предназначен для экрана сводки в клиентском приложении, где тренер может быстро оценить общую картину по своей группе, не просматривая статистику каждого спортсмена отдельно.

3.8 AI Service

AI	Service	предоставляет	публичный	endpoint
POST /api/v1/ai/athletes/{id}/recommendations			для	генерации

рекомендаций.

3.8.1 Проверка доступа

Сервис доступен только пользователю с ролью `coach`. После получения запроса AI Service обращается к User Service и проверяет, связан ли указанный спортсмен с текущим тренером. Если связи нет, возвращается ошибка доступа.

3.8.2 Получение данных

После проверки доступа сервис получает из Analytics Service:

- сводную статистику спортсмена;
- последние отчёты спортсмена.

Если отчётов нет, сервис возвращает ошибку недостатка данных. Это необходимо, потому что AI-рекомендация должна опираться на фактическую тренировочную историю, а не на пустой контекст.

3.8.3 Формирование prompt

Prompt состоит из двух частей: системного и пользовательского. Системный prompt задаёт роль LLM (помощник тренера по лёгкой атлетике) и

требует строгий формат ответа из трёх секций: «Тенденции» — наблюдения с конкретными цифрами из данных, «Риски» — сигналы тревоги, «Рекомендации» — конкретные действия на ближайшие 1–2 недели. Правила запрещают вводные фразы, вопросы к тренеру и утверждения без числового подкрепления из отчётов.

Пользовательский prompt формируется динамически и содержит общую статистику спортсмена, последние тренировки с длительностью, дистанцией, субъективной нагрузкой, показателями пульса и комментариями спортсмена, а также дополнительный контекст от тренера, если он был передан в запросе.

Количество отчётов ограничивается пятью. Это снижает размер prompt и ускоряет генерацию, сохраняя при этом достаточный контекст для анализа тенденций.

3.8.4 Генерация через Ollama

AI Service обращается к Ollama API (рисунок 12).

```
1 POST /api/chat
```

Рисунок 12 – Endpoint Ollama API для генерации текста

Используется модель `gemma3:4b` (Google Gemma 3, 4 млрд параметров). Для управления генерацией заданы параметры: `temperature: 0.7` (баланс между детерминированностью и вариативностью), `num_ctx: 2048` (размер контекстного окна в токенах) и `num_predict: 1200` (максимальная длина ответа). HTTP-таймаут вызова к Ollama составляет 5 минут, что учитывает время генерации на CPU. Ответ возвращается тренеру в виде структуры, представленной на рисунке 13.

```
1 {  
2   "athlete_id": "uuid",  
3   "type": "recommendations",  
4   "content": "Текст рекомендаций от LLM...",  
5   "generated_at": "2026-04-13T12:00:00Z",  
6   "model": "gemma3:4b"  
7 }
```

Рисунок 13 – Структура JSON-ответа AI Service

AI-модуль не заменяет тренера, а предоставляет вспомогательный текст, который тренер может учитывать при принятии решения о корректировке нагрузки. Примеры тел HTTP-запросов и ответов для основных сценариев (отправка отчёта, получение AI-рекомендации) приведены в приложении В.

3.9 Тестирование

3.9.1 Уровни тестирования

Для проверки корректности backend-платформы применяется многоуровневый подход к тестированию, который позволяет проверять систему на разных уровнях абстракции.

Unit-тесты проверяют бизнес-логику отдельных сервисов в изоляции от инфраструктуры. Зависимости заменяются mock-объектами, что позволяет быстро проверять правила доступа, обработку ошибок, граничные условия и вычисления без необходимости запуска баз данных или брокера сообщений.

E2E smoke-тест проверяет основной пользовательский сценарий через HTTP API при полностью поднятой платформе. Он проходит путь от регистрации до получения AI-рекомендации и служит для быстрой проверки того, что все сервисы запускаются и взаимодействуют корректно.

Интеграционные тесты проверяют группы endpoint через API Gateway. Они наиболее приближены к поведению реального внешнего клиента, потому что запросы проходят через тот же входной слой, включая JWT-проверку и маршрутизацию, что и мобильное приложение. Каждый запуск использует уникальные логины, что позволяет выполнять тесты повторно без очистки базы данных.

3.9.2 Unit-тесты

Unit-тесты написаны для шести сервисов и используют табличный стиль (table-driven tests) с библиотекой testify; зависимости (репозитории, NATS-публикаторы, HTTP-клиенты) заменяются mock-объектами. Тесты сервисного слоя проверяют бизнес-правила в изоляции, а тесты слоя обработчиков (auth-service, user-service) дополнительно прогоняют полный HTTP-путь через Echo с помощью net/http/httptest: разбор запроса, валидацию, вызов сервиса и формирование кодов ответа. Всего реализовано 112 unit-тестов; их распределение по сервисам приведено в таблице 17.

Таблица 17 – Распределение unit-тестов по сервисам

Сервис	Число тестов
training-service	31
user-service	28
auth-service	23
analytics-service	11
ai-service	10
notification-service	9

Содержательно unit-тесты покрывают: для auth-service — валидацию формата логина, регистрацию и дубликаты (409), вход с неверными учётными данными (401), одноразовую ротацию refresh-токена; для user-service — обязательность заголовков аутентификации (401), валидацию поиска и имени группы (400), ролевые ограничения (403); для training-service — вычисление просроченности, контроль доступа к заданиям (403/404), создание плана с назначением каждому из нескольких спортсменов, разворачивание группы в участников и сохранение как шаблон; для notification-service — регистрацию FCM-токена, пагинацию и создание уведомления из события; для analytics-service — агрегацию сводки и группировку по периодам; для ai-service — успешную генерацию, недостаток данных, ошибки Ollama и лимит отчётов в prompt.

Unit-тесты запускаются командой `make test-unit` без Docker и внешних зависимостей. Все 112 тестов шести сервисов выполняются успешно. Тестирование сознательно сосредоточено на слое бизнес-логики (сервисном) и обработчиках: слои доступа к базе данных и конфигурации проверяются на интеграционном уровне, где задействована реальная СУБД.

3.9.3 E2E и интеграционные тесты

E2E и интеграционные тесты выполняются при поднятой платформе. E2E smoke-сценарий запускается командой `make test-e2e` и проходит полный пользовательский путь из 19 шагов: от регистрации тренера и спортсмена до отправки отчёта и получения уведомлений. Интеграционные тесты запускаются командой `make test-integration` в отдельном контейнере и проверяют все группы endpoint через API Gateway — так же, как это делает

мобильное приложение, включая JWT-проверку и маршрутизацию. Каждый запуск использует уникальные логины, поэтому тесты можно выполнять повторно без очистки базы данных. Все проверки выполняются успешно.

3.9.4 Таблица результатов

Результаты всех тестовых проверок приведены в таблице 19. Таблица тестирования с критериями фиксации результатов приведена в приложении Е.

Таблица 19 – Результаты тестовых проверок

Проверка	Статус	Комментарий
<code>make test-unit</code>	112 тестов пройдено	Бизнес-логика шести сервисов
<code>make test-e2e</code>	19 шагов пройдено	Сквозной пользовательский сценарий при поднятой платформе
<code>make test-integration</code>	113 проверок пройдено	Все группы endpoint через API Gateway

3.10 Матрица трассируемости требований

После описания публичного API и уровней тестирования соответствие между функциональными требованиями, их реализацией и проверкой можно представить в явном виде. Каждое функциональное требование, сформулированное в аналитической главе, сопоставлено с реализующим его сервисом, ключевым endpoint публичного API и уровнем тестирования, на котором подтверждается его работоспособность (U — unit, I — интеграционный тест через API Gateway, E — E2E-сценарий). Результат приведён в таблице 20.

Таблица 20 – Матрица трассируемости: требование — сервис — endpoint — тест

Требование	Сервис	Ключевой endpoint	Тест
Регистрация и аутентификация, роли	Auth	POST/api/v1/auth/register, /login, /refresh	U, I, E
Разграничение прав по роли	Gateway, все сервисы	проверка JWT и роли в Gateway, заголовки X-User-Role	U, I
Заявки и связь тренер-спортсмен	User	POST/api/v1/connections/request, .../accept	U, I, E
Тренировочные группы	User	POST/api/v1/groups, .../members	U, I, E
Планы и назначения	Training	POST/api/v1/training/plans	U, I, E
Статусы и архивация назначений	Training	GET/api/v1/training/assignments	U, I
Шаблоны тренировок	Training	/api/v1/training/templates	U, I
Отчёты	Training	POST/api/v1/training/assignments/{id}/report	U, I, E
Уведомления	Notification	GET/api/v1/notifications	I, E
Аналитика	Analytics	GET/api/v1/analytics/athletes/{id}/summary	U, I
AI-рекомендации	AI	POST/api/v1/ai/athletes/{id}/recommendations	U, I

Матрица показывает, что все функциональные требования первой версии реализованы и покрыты тестами на уровне интеграционных проверок через API Gateway, а ключевые сценарии — дополнительно unit- и E2E-тестами.

3.11 Наблюдаемость

Для того чтобы работоспособность платформы можно было не только проверить тестами, но и наблюдать количественно во время работы, все семь сервисов инструментированы метриками. Инструментирование реализовано единым middleware в разделяемом Go-модуле `pkg/httpmetrics`, который подключается к каждому сервису так же, как общий пакет событий. Каждый сервис экспортирует метрики в формате Prometheus на endpoint `/metrics`.

Собираются метрики модели RED (Rate, Errors, Duration): счётчик `http_requests_total` с метками сервиса, метода, маршрута и кода ответа, а также гистограмма задержек `http_request_duration_seconds`. В качестве метки маршрута используется шаблон маршрута Echo (например, `/api/v1/training/assignments/:assignmentId`), а не сырой URL — это ограничивает кардинальность временных рядов, не давая ей расти от идентификаторов в пути. В API Gateway middleware расположен до проверки JWT, поэтому запросы, отклонённые по авторизации, также попадают в метрики.

Сбор метрик выполняет Prometheus, который опрашивает endpoint `/metrics` всех сервисов с заданным интервалом и хранит полученные временные ряды. Для визуализации в конфигурацию развёртывания включена Grafana, которая поднимается вместе с платформой и автоматически подключает Prometheus как источник данных. Такое инструментирование не предназначено для нагрузочного тестирования прототипа, а закладывает основу для эксплуатационного мониторинга при дальнейшем развёртывании платформы: по метрикам видно распределение запросов по сервисам, доля ошибок и задержки обработки.

3.12 Выводы по главе 3

В третьей главе была рассмотрена реализация backend-платформы CoachLink. Реализованы сервисы, обеспечивающие регистрацию и

авторизацию пользователей, управление профилями и связями, тренировочные группы, планы, назначения, отчёты, уведомления, аналитику и AI-рекомендации.

Система построена вокруг единого API Gateway, который проверяет JWT и маршрутизирует запросы. Межсервисное взаимодействие реализовано через HTTP и NATS JetStream. Данные разделены по сервисам, что соответствует выбранной микросервисной архитектуре.

Проведённая проверка показывает, что ключевая бизнес-логика покрыта unit-тестами, а сценарии уровня E2E и integration успешно выполняются при поднятой платформе. Это подтверждает работоспособность backend-платформы как на уровне отдельных сервисов, так и на уровне межсервисного взаимодействия через API Gateway.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была спроектирована и реализована backend-часть открытой микросервисной платформы CoachLink, предназначенной для автоматизации взаимодействия в системе «тренер-спортсмен». Разработанная система решает практическую проблему разрозненного обмена тренировочными планами и отчётами через мессенджеры, электронные таблицы и произвольные файлы, предоставляя единую серверную платформу для структурированного хранения и обработки тренировочных данных.

Целью работы была разработка backend-части открытой микросервисной платформы для автоматизации тренировочного процесса и взаимодействия в системе «тренер-спортсмен». Данная цель была достигнута за счёт проектирования и реализации набора независимых сервисов, каждый из которых отвечает за отдельную часть логики предметной области. Реализованная платформа в рамках первой версии покрывает основной цикл взаимодействия: от регистрации пользователя и установления связи между тренером и спортсменом до назначения тренировки, отправки отчёта, уведомления участников и последующего анализа данных.

В ходе выполнения работы были последовательно решены все поставленные задачи. В аналитической главе проведён обзор существующих способов организации тренировочного процесса. Показано, что мессенджеры, электронные таблицы, спортивные трекеры и специализированные коммерческие платформы решают отдельные части проблемы, но не предоставляют одновременно единый формат отчётности, поддержку ролей, групповые назначения, открытую архитектуру и возможность самостоятельного развёртывания. На основе анализа сформулированы функциональные и нефункциональные требования к backend-платформе.

В проектной части предложена микросервисная архитектура, в которой функциональность разделена между API Gateway, Auth Service, User Service, Training Service, Notification Service, Analytics Service и AI Service. Для межсервисного взаимодействия используются синхронные HTTP-вызовы и событийная коммуникация через NATS JetStream. Спроектирована модель данных с разделением баз по сервисам, определены ключевые пользовательские сценарии и описан публичный REST API в формате

OpenAPI.

В главе реализации описана практическая разработка backend-сервисов на языке Go. Каждый сервис имеет собственную область ответственности и собственную схему данных. Stateless-сервисы, такие как Analytics Service и AI Service, работают как вычислительные модули поверх данных других сервисов. Проведено тестирование на трёх уровнях: 112 unit-тестов бизнес-логики шести сервисов, E2E smoke-сценарий из 19 шагов и интеграционные тесты через API Gateway. Все тестовые проверки выполняются успешно, что подтверждает работоспособность как отдельных сервисов, так и межсервисного взаимодействия. Соответствие реализованных функций сформулированным требованиям прослеживается по матрице трассируемости (требование — endpoint — тест), приведённой в главе реализации: реализованы поддержка ролей пользователей, управление связями тренера и спортсмена, тренировочные группы, назначения и отчёты, уведомления, аналитика и AI-рекомендации. Дополнительно платформа инструментирована метриками в формате Prometheus, что делает работу сервисов наблюдаемой во время эксплуатации.

Практическая значимость работы заключается в том, что разработанный backend предоставляет тренеру единый механизм назначения тренировок и отслеживания их выполнения, а спортсмену — стандартизированный формат отчёта, пригодный для последующего анализа. Платформа формирует централизованную историю тренировочного процесса, поддерживает систему уведомлений о ключевых событиях и содержит вспомогательный AI-модуль для формирования рекомендаций по последним отчётам спортсмена. Благодаря открытому API система может быть использована различными клиентскими приложениями без привязки к конкретному пользовательскому интерфейсу.

Архитектурным результатом работы является разделение системы на независимые сервисы с чёткими зонами ответственности. Такое разделение повышает сопровождаемость: изменение механизма уведомлений не затрагивает сервис тренировок, развитие аналитики не влияет на авторизацию, а расширение AI-сценариев может выполняться отдельно от базового цикла назначений и отчётности.

Текущая реализация представляет собой прототип backend-платформы. В рамках работы не разрабатывалось мобильное приложение, а предметная

модель ориентирована на лёгкую атлетику. AI-модуль реализован как интеграция с локальным LLM-сервером и предоставляет вспомогательные рекомендации на основе тренировочной истории, не являясь самостоятельной AI-разработкой.

Дальнейшее развитие платформы может включать поддержку других видов спорта за счёт расширения модели тренировочных планов и отчётов, интеграцию со спортивными устройствами для автоматического заполнения части отчёта, развитие аналитики с анализом динамики нагрузки и выявлением тенденций, а также расширение AI-сценариев. Предметная реализация первой версии ориентирована на лёгкую атлетику, однако архитектура платформы спроектирована с учётом этих направлений и не требует полного перепроектирования для их реализации.

Поставленная цель достигнута, а задачи выпускной квалификационной работы выполнены в полном объёме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Richardson C. Microservices Patterns: With Examples in Java. — Shelter Island : Manning Publications, 2018. — ISBN 978-1617294549.
2. Newman S. Building Microservices: Designing Fine-Grained Systems. — 2-е изд. — Sebastopol : O'Reilly Media, 2021. — ISBN 978-1492034025.
3. Microservices: Yesterday, Today, and Tomorrow / N. Dragoni [и др.] // Present and Ulterior Software Engineering. — Cham : Springer, 2017. — С. 195—216. — DOI: 10.1007/978-3-319-67425-4_12.
4. Balalaie A., Heydarnoori A., Jamshidi P. Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture // IEEE Software. — 2016. — Т. 33, № 3. — С. 42—52. — DOI: 10.1109/MS.2016.64.
5. The Go Programming Language Documentation / The Go Authors. — URL: <https://go.dev/doc/> (дата обращения 14.12.2025).
6. Echo Web Framework Documentation / LabStack. — URL: <https://echo.labstack.com/docs> (дата обращения 09.01.2026).
7. PostgreSQL Documentation / PostgreSQL Global Development Group. — URL: <https://www.postgresql.org/docs/> (дата обращения 22.12.2025).
8. RFC 7519: JSON Web Token (JWT) / Internet Engineering Task Force. — URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата обращения 15.01.2026).
9. NATS Documentation / Synadia Communications. — URL: <https://docs.nats.io/> (дата обращения 05.02.2026).
10. Docker Compose Documentation / Docker Inc. — URL: <https://docs.docker.com/compose/> (дата обращения 03.12.2025).
11. Ollama Documentation / Ollama. — URL: <https://github.com/ollama/ollama> (дата обращения 18.03.2026).
12. OpenAPI Specification / OpenAPI Initiative. — URL: <https://spec.openapis.org/oas/latest.html> (дата обращения 27.01.2026).

ПРИЛОЖЕНИЕ А

Состав backend-сервисов

В приложении приведена таблица с кратким описанием сервисов. Она позволяет наглядно показать состав backend-платформы и зону ответственности каждого сервиса.

Таблица А.1 – Состав backend-сервисов CoachLink

Сервис	Назначение	Хранилище	Основные взаимодействия
API Gateway	Единая точка входа, JWT-проверка, reverse proxy	Без собственной БД	Принимает внешние REST-запросы и маршрутизирует их на внутренние сервисы
Auth Service	Регистрация, login, refresh/logout, выдача JWT	PostgreSQL	Публикует <code>coachlink.user.registered</code> в NATS
User Service	Профили, заявки, связи, группы, поиск	PostgreSQL	Принимает событие регистрации, предоставляет внутренние проверки доступа
Training Service	Планы, шаблоны, назначения, отчёты, статусы	PostgreSQL	Публикует события назначений и отчётов, предоставляет данные аналитике
Notification Service	Уведомления, unread count, FCM-токены	PostgreSQL	Подписывается на события NATS
Analytics Service	Summary, progress, coach overview	Stateless	Получает данные из Training Service и User Service
AI Service	Рекомендации по последним отчётам спортсмена	Stateless	Проверяет связь через User Service, получает данные из Analytics Service, обращается к Ollama

AI Service и Analytics Service реализованы как stateless-сервисы. Они не владеют собственными предметными данными и формируют результат на основе данных других сервисов.

ПРИЛОЖЕНИЕ Б

Группы публичного API

В приложении представлена таблица публичных API-групп. Полный перечень endpoints содержится в docs/api/openapi.yaml, а в тексте диплома используется сгруппированное представление.

Таблица Б.1 – Группы публичного API и их назначение

Группа API	Примеры endpoints	Назначение
Auth API	POST /api/v1/auth/register, POST /api/v1/auth/login	Регистрация и аутентификация пользователей
User API	/api/v1/users/me, /api/v1/users/search	Профили, поиск, заявки, связи, группы
Training API	/api/v1/training/plans, /api/v1/training/ assignments/{id}/report	Планы, назначения, отчёты и шаблоны
Notification API	/api/v1/notifications, /api/v1/notifications/{id}/ read	Уведомления и отметка прочтения
Analytics API	/api/v1/analytics/athletes/ {id}/summary, /api/v1/analytics/coach/ overview	Агрегация тренировочных данных
AI API	POST /api/v1/ai/athletes/ {id}/recommendations	Рекомендации по последним отчётам спортсмена
System API	/health, /api/v1/health	Проверка доступности сервисов

ПРИЛОЖЕНИЕ В

Примеры JSON-запросов и ответов

Примеры API отражают backend-логику и не зависят от особенностей мобильного интерфейса. В качестве репрезентативных примеров приведены отправка отчёта и получение AI-рекомендации.

Пример отправки отчёта о тренировке

```
1 {
2   "content": "Выполнил кросс спокойно, последние 2 км немного
3     быстрее.",
4   "duration_minutes": 46,
5   "perceived_effort": 6,
6   "max_heart_rate": 174,
7   "avg_heart_rate": 151,
8   "distance_km": 8.0
9 }
```

Пример показывает тело запроса для endpoint POST /api/v1/training/assignments/{assignmentId}/report.

Идентификатор назначения передаётся в path-параметре, а отчёт хранится в структурированном виде. В отличие от сообщения в мессенджере, backend может использовать поля отчёта для аналитики: рассчитывать недельный объём, длительность, дистанцию, среднюю воспринимаемую нагрузку и пульсовые показатели.

Пример ответа на создание отчёта

```
1 {
2   "id": "7f9ef6c3-0fd9-4cfd-a69d-d3d1b65a157d",
3   "assignment_id": "2b54c487-0ef3-4e60-8e9a-7f2fdb66c101",
4   "athlete_id": "a52d4f18-e218-4d2a-bd4f-01bcb89d01f2",
5   "athlete_full_name": "Петров Иван Сергеевич",
6   "athlete_login": "ivan-petrov",
7   "content": "Выполнил кросс спокойно, последние 2 км немного
8     быстрее.",
9   "duration_minutes": 46,
10  "perceived_effort": 6,
11  "max_heart_rate": 174,
```

```

11  "avg_heart_rate": 151,
12  "distance_km": 8.0,
13  "created_at": "2026-04-18T18:30:00Z"
14  }

```

После создания отчёта Training Service публикует событие, на которое подписан Notification Service. Благодаря этому тренер получает уведомление без прямой зависимости между Training Service и способом доставки уведомлений.

Пример запроса AI-рекомендации

```

1  {
2    "context": "Подготовка к старту на 800 м через два месяца."
3  }

```

Пример ответа AI Service

Ответ возвращается в формате JSON. Поле content содержит фактический текст, сгенерированный моделью gemma3:4b по реальной истории тренировок тестового спортсмена.

```

1  {
2    "athlete_id": "a52d4f18-e218-4d2a-bd4f-01bcb89d01f2",
3    "type": "recommendations",
4    "content": "
5    Тенденции
6    Средняя продолжительность тренировок — 47 мин, дистанция в
    последней сессии выросла до 6.4 км против стабильных 5.0 км в
    предыдущих двух. Воспринимаемая нагрузка — 8/10 в каждой из
    трёх последних тренировок, максимальный пульс 182 удмин/
    зафиксирован в интервальной сессии.
7
8    Риски
9    Сочетание высокой субъективной нагрузки (8/10) с ростом объёма и
    комментарием спортсмена «тяжёлые» ноги на последних отрезках»
    указывает на накопленную усталость. Пульс 182 удмин/ при 100%
    выполнения заданий — вероятный признак недовосстановления.
10
11   Рекомендации
12   1. Следующую интервальную сессию сократить до 6×400 м и провести

```

```
13     не ранее чем через 72 часа.  
14 2. Добавить один восстановительный день с лёгким бегом до 30 мин  
15    при пульсе ниже 140.  
16 3. Удерживать пульс на отрезках ниже 178 удмин/ для контроля  
17    интенсивности.  
18 Модель: gemma3:4b · локально через Ollama · данные: 3 последних  
19    отчёта тестового спортсмена  
20    ",  
21    "generated_at": "2026-04-18T18:35:00Z",  
22    "model": "gemma3:4b"  
23 }
```

AI-рекомендация является вспомогательной информацией. Она не заменяет решение тренера и не используется как автоматическое назначение тренировочного плана.

ПРИЛОЖЕНИЕ Г

ER-диаграмма основных таблиц

В приложении приведена расширенная ER-диаграмма, отражающая структуру и связи основных таблиц backend-платформы. В отличие от диаграммы в основном тексте, данное представление включает детализацию полей и типов данных. В микросервисной архитектуре CoachLink таблицы физически распределены по базам данных отдельных сервисов. На рисунке Г.1 представлена расширенная ER-диаграмма.

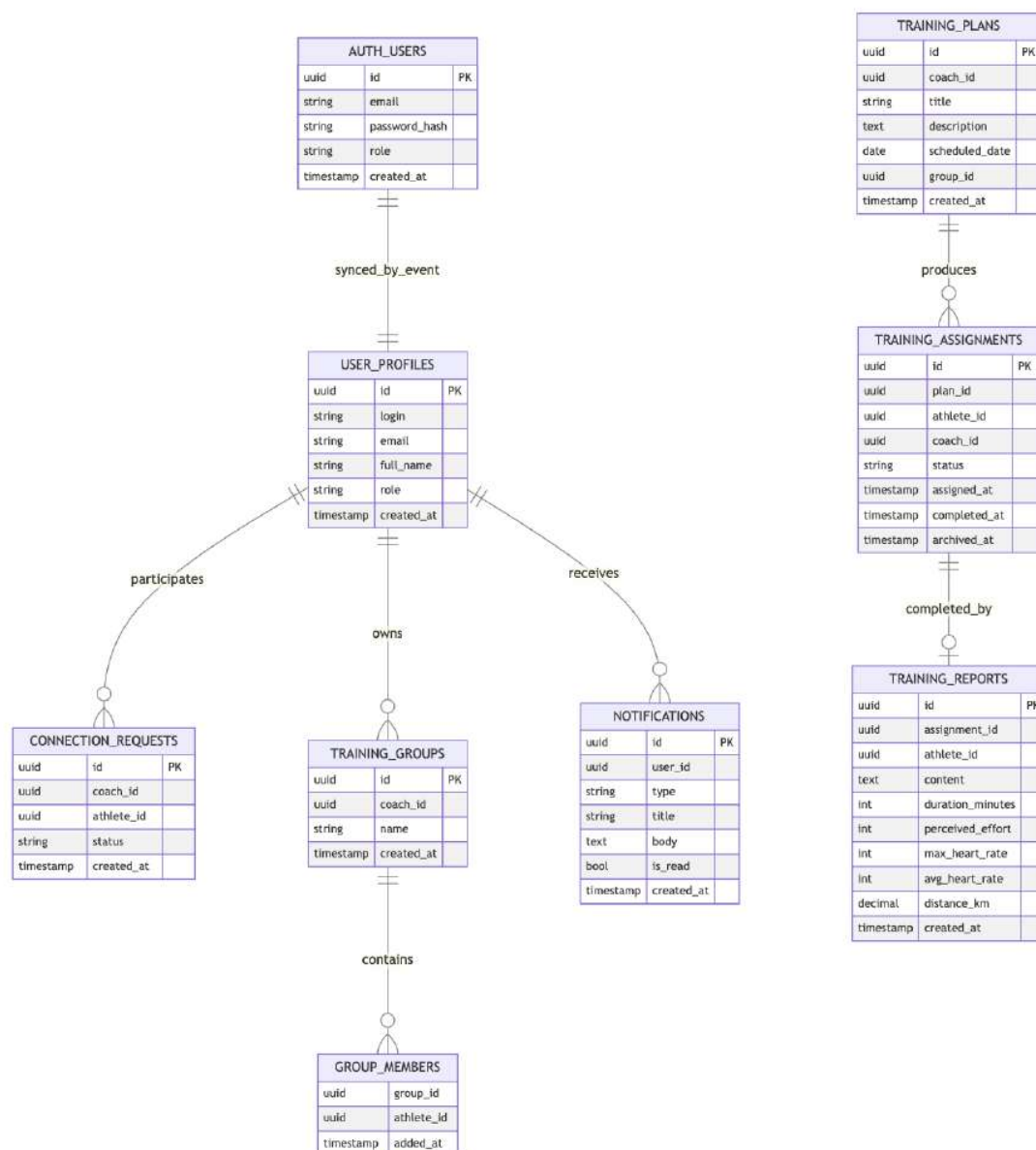


Рисунок Г.1 – Расширенная ER-диаграмма основных таблиц платформы CoachLink

ПРИЛОЖЕНИЕ Д

События NATS JetStream в backend-платформе

В приложении приведена таблица событий, используемых для слабосвязанного взаимодействия сервисов.

Таблица Д.1 – События NATS JetStream для межсервисного взаимодействия

Событие	Источник	Получатель	Назначение
<code>coachlink.user.registered</code>	Auth Service	User Service	Создание профиля пользователя
<code>coachlink.connection.requested</code>	User Service	Notification Service	Уведомление тренера о новой заявке
<code>coachlink.connection.accepted</code>	User Service	Notification Service	Уведомление спортсмена о принятии заявки
<code>coachlink.connection.rejected</code>	User Service	Notification Service	Уведомление спортсмена об отклонении заявки
<code>coachlink.group.athlete_added</code>	User Service	Notification Service	Уведомление о добавлении в группу
<code>coachlink.group.athlete_removed</code>	User Service	Notification Service	Уведомление об удалении из группы
<code>coachlink.training.assigned</code>	Training Service	Notification Service	Уведомление о назначенной тренировке
<code>coachlink.training.deleted</code>	Training Service	Notification Service	Уведомление об удалении назначения
<code>coachlink.report.submitted</code>	Training Service	Notification Service	Уведомление тренера о новом отчёте

Названия событий взяты из общего пакета `pkg/events` и соответствуют текущей событийной модели платформы.

ПРИЛОЖЕНИЕ Е

Таблица тестирования

Таблица тестирования отражает фактически подтверждённые результаты запусков.

Таблица Е.1 – Уровни тестирования и критерии фиксации результатов

Уровень тестирования	Команда	Назначение	Статус
Unit-тесты	make test-unit	Проверка сервисного слоя отдельных микросервисов	112 тестов пройдено
E2E smoke	make test-e2e	Проверка базового e2e-сценария при поднятой платформе	19 шагов пройдено
Integration	make test-integration	Проверка API Gateway и межсервисных сценариев	113 проверок пройдено